



Dolda Markovmodeller och statistisk inferens för smittspridning på en sjukhusavdelning

Hidden Markov Models and Statistical Inference for Infection Spread in a Hospital Ward

Examensarbete för kandidatexamen i matematik vid Göteborgs universitet

Kandidatarbete inom civilingenjörsutbildningen vid Chalmers

Lukas Gustafsson

Hanna Isaksson

Ellinor Olesen

Einar Schoon

Dolda Markovmodeller och statistisk inferens för smittspridning på en sjukhusavdelning

Examensarbete för kandidatexamen i matematik inom Matematikprogrammet, inriktning Tillämpad matematik, vid Göteborgs universitet

Lukas Gustafsson

Kandidatarbete i matematik inom civilingenjörsprogrammet Automation och mekatronik vid Chalmers

Hanna Isaksson

Kandidatarbete i matematik inom civilingenjörsprogrammet Teknisk matematik vid Chalmers

Ellinor Olesen Einar Schoon

Handledare: Oskar Holmstedt

Institutionen för Matematiska vetenskaper
CHALMERS TEKNISKA HÖGSKOLA
GÖTEBORGS UNIVERSITET
Göteborg, Sverige 2026

Förord

Denna rapport utgör vårt kandidatarbete vid Chalmers tekniska högskola och Göteborgs universitet. Vi vill tacka vår handledare Oskar Holmstedt för hans hjälp och vägledning under projektet.

Under projektets gång har det veckovis förts loggar över hur projektet fortskred. Detta gjordes med ett rullande schema där varje medlem hade ansvar för dagboken var fjärde vecka. Nedan följer en beskrivning av varje enskild medlems bidrag samt en bidragsrapport som illustrerar vem som skrivit vad i rapporten.

Gruppens tillvägagångssätt

Arbetet inleddes med att samtliga gruppmedlemmar läste in sig på ämnet. Under hela projektets gång höll gruppen regelbundna veckomöten där problem, resultat och kommande arbetsmoment diskuterades. En del av arbetet skedde gemensamt, där gruppen tillsammans främst arbetade med rapportskrivning och diskussion av modellen.

Lukas: Har haft ansvar för skrivandet av inledningen och algoritmdelen i teorikapitlet. Har även ansvarat för att referenser och källor är korrekt hanterade i rapporten samt skrivit stora delar av avsnittet om samhällseliga och etiska aspekter.

Hanna: Har skrivit mycket kod för testerna samt haft ansvar för sammandraget, syftet, avgränsningarna, resultatet och slutsatsen i rapporten. Har deltagit i diskussion av resultaten för både infektiviteten och testningsgraden.

Ellinor: Har haft ansvar för hur modellen ska se ut både teoretiskt och programmeringsmässigt. Har tillsammans med Hanna skrivit kod för testningsgradstester och sammanställt all kod. Har även gett många kommentarer på andras texter och säkerställt att teorin och pseudokoden för algoritmerna stämmer. Har dessutom haft en mer övergripande roll i projektet och ansvarat för administrativa delar. Hade lite överblick om vad som skulle respektive inte skulle vara med i diskussionen och diskuterade resultaten för testerna samt modellens svagheter. Har också haft ansvar för metoddelen.

Einar: Har ansvarat för att sammanställa teoridelen och säkerställa dess korrekthet och sammanhang. Har även varit delaktig i att skriva pseudokoden. Har skrivit kod för infektivitetsanalysen tillsammans med Hanna samt haft ett särskilt ansvar för resultatdelen kopplad till infektiviteten. Har bidragit till att upprätthålla en konsekvent terminologi och struktur genom hela rapporten. Har deltagit i diskussion av resultaten för både infektiviteten, testningsgraden och modell, samt sammanställt den slutliga diskussionen. Har även ansvarat för bakgrunden.

Bidragsrapport		
-	Rubrik	Författarna
-	Förord	Ellinor
-	Sammandrag	Hanna
-	Abstract	Hanna
1	Inledning	Lukas och Einar
1.1	Syfte	Hanna
1.2	Avgränsningar	Hanna
1.3	Bakgrund	Einar
2	Teori	Einar
2.1	Dolda Markovmodeller	Einar
2.2	Algoritmer för inferens	Lukas och Ellinor
2.3	Modellformulering	Einar
3	Metod	Ellinor
3.1	Testningsgradens påverkan	Ellinor
3.2	Infektivitetens påverkan	Ellinor
4	Resultat	Einar, Ellinor och Hanna
4.1	Testningsgradens påverkan	Hanna
4.2	Infektivitetens påverkan	Einar
5	Diskussion	Einar
5.1	Samhälleliga och etiska aspekter	Lukas och Ellinor
5.2	Slutsatser	Hanna
A	Källkod	Ellinor
-	Skriva kod för modellen	Ellinor
-	Skriva kod för tester	Ellinor, Hanna och Einar

Sammandrag

Denna rapport undersöker genom en simuleringsstudie hur väl smittspridningen på en mindre sjukhusavdelning kan rekonstrueras utifrån begränsade observationer, med hjälp av en dold Markovmodell. Modellen beskriver patienters smittstatus som dolda tillstånd och testresultat som stokastiska observationer. Inferensalgoritmerna Viterbi och forward-backward används för att estimerade de underliggande tillstånden, där komponentvisa och tillståndsvisa noggrannhetsmått infördes. I studien analyseras hur rekonstruktionens noggrannhet påverkas av testningsgraden och förhållandet mellan väntevärdet för smittotid och väntevärdet för utskrivningstid. Testningsgraden varierades för att undersöka påverkan på noggrannheten och infektiviteten analyserades genom att variera väntevärdet för smittotiden med väntevärdet för utskrivningstiden fixerad, över flertalet simuleringar. Resultatet visar att noggrannheten ökar med högre testningsgrad för samtliga algoritmer, vilket var förväntat. Den komponentvisa noggrannheten för Viterbi-algoritmen och forward-backward-algoritmen var hög redan vid låg testningsgrad, medan Viterbi-algorithmens tillståndsvisa noggrannhet var sämre och krävde högre testningsgrad för att uppnå motsvarande noggrannhet. Analysen visade att Viterbi-algorithmens komponentvisa noggrannhet var högre när tillståndssekvensen övergick till tillstånd med samtliga okoloniserade patienter eller vid fullständig kolonisering. Den lägsta noggrannheten uppvisades i området där väntevärdena för smittotiden och för utskrivningstiden var av liknande magnitud. Gällande samhälleliga och etiska aspekter noteras eventuella komplikationer vid insamling och användning av verklig patientdata, eftersom de klassas som känsliga, och att modellens verklighetsbaserings är förenklad. Potentiella utvecklingsområden är implementering av verklig tid istället för diskreta tidssteg, väsentlig information gällande patienter och heterogenitet.

Abstract

This report investigates, through a simulation study, how well the spread of infection in a small hospital ward can be reconstructed from limited observations using a hidden Markov model. The model describes patients' infection status as hidden states and test results as stochastic observations. The inference algorithms Viterbi and forward-backward are used to estimate the underlying states, where component-wise and state-wise accuracy measures are introduced. The study analyzes how the reconstruction accuracy is affected by the testing rate and the relationship between the expected infection time and the expected discharge time. The testing rate is varied to examine its effect on accuracy, and the infectivity is analyzed by varying the expected infection time while keeping the expected discharge time fixed, across multiple simulations. The results show that the accuracy increases with higher testing rates for all algorithms, which was expected. The component-wise accuracy of both the Viterbi algorithm and the forward-backward algorithm is high even at low testing rates, while the state-wise accuracy of the Viterbi algorithm is lower and requires higher testing rates to achieve similar accuracy. The analysis also shows that the component-wise accuracy of the Viterbi algorithm is higher when the state sequence transitions to states where all patients are uncolonized or when full colonization occurs. The lowest accuracy is observed in the region where the expected infection time and the expected discharge time were of similar magnitude. Regarding societal and ethical aspects, possible challenges are noted when collecting and using real patient data, as such data are classified as sensitive, and that the model relies on simplifying assumptions. Potential areas for further development include to implement actual time intervals between events instead of discrete time steps and to take patient-specific information and heterogeneity into account.

Innehåll

1	Inledning	1
1.1	Syfte	1
1.2	Avgränsningar	1
1.3	Bakgrund	2
2	Teori	3
2.1	Dolda Markovmodeller	3
2.2	Algoritmer för inferens	4
2.2.1	Forward-backward-algoritmen	4
2.2.2	Viterbi-algoritmen	6
2.3	Modellformulering	7
2.3.1	Markovkedjan	7
2.3.2	Emissionsmatris	8
2.3.3	Väntevärde av a posteriorifördelningen	9
3	Metod	9
3.1	Testningsgradens påverkan	9
3.2	Infektivitetens påverkan	10
4	Resultat	11
4.1	Testningsgradens påverkan	12
4.1.1	Viterbi-algoritmen	12
4.1.2	Forward-backward-algoritmen	13
4.2	Infektivitetens påverkan	14
5	Diskussion	15
5.1	Samhälleliga och etiska aspekter	16
5.2	Slutsats	16
A	Källkod	i
A.1	Generera övergångsmatris	i
A.2	Markovkedja	iv
A.3	Dold Markovkedja	vi
A.4	Tester	xii

1 Inledning

Infektionssjukdomar har genom alla tider varit en stor bidragande orsak till global dödlighet. För endast några hundra år sedan uppskattas infektioner ha orsakat mer än hälften av alla dödsfall [1]. Först under det senaste århundradet har kroniska sjukdomar istället blivit den främsta dödsorsaken. Denna övergång är en del av vad som brukar kallas den epidemiologiska övergången. Trots detta dör fortfarande miljontals människor globalt till följd av just infektionssjukdomar [2]. Dessutom har ett nytt problem tillkommit i form av antibiotikaresistenta bakterier [3]. Den ökande resistensen betraktas nu som ett stort hot mot folkhälsan.

I sjukhusmiljöer är situationen gällande smittspridning särskilt utsatt. Där vårdas ofta många patienter samtidigt, och vissa är särskilt sårbara. Infektioner som inträffar inom vården kallas nosokomiala infektioner och för att en infektion ska klassas som nosokomial ska symptomen framträda tidigast 48 timmar efter inskrivning eller inom några dagar efter utskrivning [4]. Varje år drabbas uppskattningsvis 4,3 miljoner patienter inom EU av nosokomiala infektioner [5], och enbart i Sverige drabbas årligen mer än 50 000 patienter av vårdrelaterade infektioner, med en genomsnittlig kostnad på 107 000 kronor per infektion för svensk sjukvård [6]. Detta understryker vikten av att förstå hur smittspridningen kan begränsas i dessa miljöer.

Den historiska utvecklingen visar att infektionssjukdomar är ett område där framsteg inom forskning inte bara behövs, utan också är möjliga. För att minska smittspridningen behövs effektiva metoder för att analysera hur smittor sprids över tid och i olika sammanhang. Matematisk modellering har därför blivit ett viktigt verktyg inom epidemiologisk forskning. Särskilt under det senaste århundradet har sådana modeller utvecklats, bland annat efter att Kermack och McKendrick introducerade den klassiska SIR-modellen på 1920-talet [7].

Mot denna bakgrund studeras i denna studie smittspridning på en intensivvårdsavdelning med hjälp av matematisk modellering. En central svårighet i sammanhanget är att patienters faktiska smittstatus inte alltid är direkt observerbar. För att hantera problemet används i denna studie dolda Markovmodeller, där smittstatusen hos patienterna betraktas som en dold Markovkedja och de indirekta observationerna består av slumpmässiga tester av patienter. Utifrån dessa observationer kan den dolda smittspridningsprocessen skattas med hjälp av inferensalgoritmer.

1.1 Syfte

Studiens syfte är att genom en simuleringsstudie undersöka hur väl det går att rekonstruera inlagda patienters tillstånd, baserat på begränsade stokastiska observationer på en mindre sjukhusavdelning. Genom återskapande av händelseförloppet kan ökad förståelse för den underliggande spridningsprocessen uppnås. Detta genomförs genom implementering av inferensalgoritmer, som forward-backward-algoritmen och Viterbi-algoritmen. Följande frågeställningar besvaras under projektets gång:

- Hur påverkar testningsgraden resultatet?
- Hur påverkas resultatet av förhållandet mellan tidsväntevärdet till utskrivning och tidsväntevärdet till smitta?

1.2 Avgränsningar

Studien avgränsas till att analysera en mindre sjukhusavdelning på tio sängar. Detta görs eftersom modellens komplexitet för antalet möjliga tillstånd växer exponentiellt med antalet patienter, där varje enskild individ befinner sig i ett av två tillstånd, okoloniserad (osmittad) eller koloniserad (smittad). En alltför stor population leder till att modellens komplexitet blir svårhanterlig rent beräkningsmässigt och därmed tidskrävande vid simulering. Det är av intresse att undersöka mindre avdelningar eftersom det är viktigt att förstå hur smittspridningen utvecklas i sådana miljöer, exempelvis på en intensivvårdsavdelning med begränsad kapacitet. Ingen verklig data används i studien utan endast syntetisk data från simuleringar.

1.3 Bakgrund

Matematisk modellering av smittspridning i sjukhusmiljöer har förändrats och utvecklats över tid. I en systematisk genomgång av 96 studier om matematisk modellering av vårdrelaterade infektioner fann van Kleef et al. att antalet publikationer på området ökade stadigt under perioden 1993–2011 [8]. Genomgången visar även att andelen studier som använder stokastiska modeller har ökat över tid, medan andelen deterministiska modeller har minskat. Vidare använde en majoritet av studierna en modell där populationen delas in i grupper baserat på smittstatus, snarare än att följa enskilda individers smittstatus. Den individbaserade metoden har fördelen att heterogenitet kan modelleras, det vill säga att olika individer smittas eller blir smittade med olika sannolikhet, men detta sker på bekostnad av ökad beräkningsbörda.

Genomgången [8] lyfter även fram de olika typer av smittskyddsåtgärder som modellerats, där ibland handhygien, isolering, kohortering och screening. Där screening, det vill säga aktiv testning av patienter för att identifiera smittbärare, är av särskilt intresse för detta projekt. Screening studerades i 18 procent av de interventionsstudier som ingick i genomgången. Utöver utvärdering av interventioner framhålls också att modeller har använts för att öka den epidemiologiska förståelsen av smittspridningsprocessen, bland annat genom att skatta smittspridningsparametrar från data. En central utmaning vid modellering av nosokomiala infektioner är att smittade patienter ofta bär på patogenen utan att uppvisa några symptom, och att en symptomatisk infektion endast utvecklas hos en andel av de smittade patienterna [9]. Detta innebär att den underliggande smittspridningsprocessen vanligtvis endast är delvis observerbar, vilket försvårar statistisk inferens men motiverar användningen av dolda Markovmodeller.

2 Teori

En sekvens av smittspridningsdata på en sjukhusavdelning utgörs inte av direkt observation av den underliggande smittspridningen. I stället består den av observationer som beror på tillståndet i den faktiska smittspridningsprocessen. För att beskriva denna typ av struktur kan en dold Markovmodell användas [9]. I följande avsnitt introduceras modellerna som utgör grunden för modellformuleringen i studien.

2.1 Dolda Markovmodeller

Dolda Markovmodeller bygger på att det finns en stokastisk process som modelleras av en Markovkedja. Denna process antas vara dold för direkt observation men kan observeras indirekt genom en stokastisk process som beror på den dolda processen [10][11]. I studien begränsas Markovkedjan i den dolda Markovmodellen till att vara en homogen diskret Markovkedja. Denna typ av kedja är en stokastisk process $X = \{X_t : t \in \mathbb{N}\}$, med ett ändligt tillståndsrum $\mathcal{S}$, som uppfyller

$$\begin{aligned} \mathbb{P}(X_n = x_j \mid X_0 = x_0, X_1 = x_1, \dots, X_{n-1} = x_i) &= \mathbb{P}(X_n = x_j \mid X_{n-1} = x_i) \\ &= \mathbb{P}(X_1 = x_j \mid X_0 = x_i), \\ \forall n \geq 1, \forall x_i, x_j, x_0, x_1, \dots, x_{n-2} \in \mathcal{S}. \end{aligned} \quad (1)$$

Sannolikheterna (1) kallas övergångssannolikheter och kan ordnas i en $|\mathcal{S}| \times |\mathcal{S}|$ -matris P kallad övergångsmatrisen, där $P_{ij} = \mathbb{P}(X_1 = j \mid X_0 = i)$. För att fullständigt specificera en Markovkedja krävs även en initialfördelning $\pi \in \mathbb{R}^{|\mathcal{S}|}$, som definieras av $\pi_i = \mathbb{P}(X_0 = i)$ [12].

Observationerna av den dolda processen modelleras av en stokastisk process $O = \{O_t : t \in \mathbb{N}\}$ med ett ändligt tillståndsrum Ω . För varje $x_i \in \mathcal{S}$ och $o_j \in \Omega$ definieras emissionssannolikheterna som

$$E_{ij} = \mathbb{P}(O_t = o_j \mid X_t = x_i). \quad (2)$$

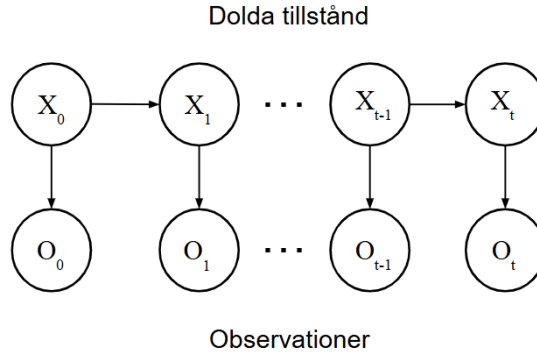
Dessa sannolikheter kan ordnas i en $|\mathcal{S}| \times |\Omega|$ -matris E , kallad emissionsmatrisen. En dold Markovmodell antas ha en beroendestruktur sådan att den gemensamma fördelningen för den dolda processen och observationsprocessen faktoriseras som

$$\begin{aligned} \mathbb{P}(X_t = x_t, X_{t-1} = x_{t-1}, \dots, X_0 = x_0, O_t = o_t, O_{t-1} = o_{t-1}, \dots, O_0 = o_0) \\ = \mathbb{P}(X_0 = x_0) \prod_{i=1}^t \mathbb{P}(X_i = x_i \mid X_{i-1} = x_{i-1}) \prod_{i=0}^t \mathbb{P}(O_i = o_i \mid X_i = x_i) \end{aligned} \quad (3)$$

[11]. Av antagandet i (3) följer att varje observation O_t endast beror på det dolda tillståndet X_t vid varje tidpunkt, det vill säga

$$\mathbb{P}(O_t = o_t \mid X_0 = x_0, \dots, X_t = x_t, O_0 = o_0, \dots, O_{t-1} = o_{t-1}) = \mathbb{P}(O_t = o_t \mid X_t = x_t). \quad (4)$$

Den dolda Markovmodellen bestäms därmed av övergångsmatrisen P , initialfördelningen π , emissionsmatrisen E samt beroendestrukturen i (3). En grafisk illustration av modellen ges i figur 1.



Figur 1: Markovkedja med dolda tillstånd och tillhörande observationer över tid.

2.2 Algoritmer för inferens

I denna studie används två centrala algoritmer för inferens i dolda Markovmodeller: forward-backward-algoritmen och Viterbi-algoritmen [10]. Båda algoritmerna använder en sekvens av observationer för att dra slutsatser om den underliggande dolda tillståndsprocessen, men de gör detta på olika sätt. Syftet med forward-backward-algoritmen, beskriven i algoritm 1, är att beräkna sannolikhetsfördelningen över alla potentiella dolda tillstånd vid varje tidpunkt, givet en sekvens av observationer. Viterbi-algoritmen, som beskrivs i algoritm 2, används istället för att finna den mest sannolika sekvensen av de dolda tillstånden, givet en sekvens av observationer. I båda algoritmerna betecknas antal tillstånd n och tidindexering börjar vid 0, där första observationen betecknas y_0 .

2.2.1 Forward-backward-algoritmen

Forward-backward-algoritmen är en metod för inferens i dolda Markovmodeller [10][11]. Algoritmen förutsätter att modellens parametrar är kända. Dessa utgörs av övergångssannolikheter P_{lk} , emissionssannolikheter $E_{k,y_t} = \mathbb{P}(O_t = y_t \mid X_t = x_k)$ samt startfördelningen π . Variabeln y_t betecknar observationen vid tidpunkt t , givet det dolda tillståndet x_k . Den fullständiga algoritmen ges i algoritm 1. För att underlätta förståelsen beskrivs här först forward- och backward-algoritmerna separat. Därefter beskrivs den kombinerade algoritmen, som används för att beräkna den fullständiga sannolikhetsfördelningen.

Forward-algoritmen

Sannolikheten $\alpha_k(t) = \mathbb{P}(X_t = x_k, O_t = y_t, \dots, O_0 = y_0)$ kallas forward-variabeln och beräknas i algoritm 1 med rekursionen

$$\alpha_k(t) = E_{k,y_t} \sum_{l=1}^n \alpha_l(t-1) P_{lk}$$

[11][10]. Termerna i produkten består av emissionssannolikheten och summan över tidigare tillstånd x_l . Produkten i summan motsvarar sannolikheten att observationerna fram till tidpunkt $t-1$ har observerats och att systemet då befinner sig i tillstånd x_l , följt av en övergång till tillstånd x_k . Rekursionen i forward-algoritmen ger sannolikheten för att observationerna fram till tidpunkt t har inträffat och att systemet befinner sig i tillstånd x_k vid tidpunkt t .

Backward-algoritmen

Sannolikheten $\beta_k(t) = \mathbb{P}(O_{t+1} = y_{t+1}, \dots, O_T = y_T \mid X_t = x_k)$ kallas backward-variabeln och beräknas i algoritm 1 baklänges rekursivt med

$$\beta_k(t) = \sum_{l=1}^n E_{l,y_{t+1}} \beta_l(t+1) P_{kl}$$

[11][10]. Det som beräknas i rekursionen är sannolikheten för de återstående observationerna från tidpunkt $t+1$ till T , givet att modellen befinner sig i tillstånd x_k vid tidpunkt t , med hjälp av emissionssannolikheter, övergångssannolikheter och tidigare beräknad sannolikhet $\beta_l(t+1)$.

Algorithm 1 Forward-backward-algoritmen [11] [10]

Förutsättning: Observationssekvens $\mathbf{y}$, startfördelning $\boldsymbol{\pi}$, övergångsmatris P samt emissionsmatris E .

Resultat: $\mathbb{P}(X_t = x_k \mid \mathbf{y})$ för alla t, k .

1. Forward-algoritmen

1.1 Initialvärden

```
1: for  $k = 1, \dots, n$   
2:    $\alpha_k(0) \leftarrow \boldsymbol{\pi} \cdot E_{k, y_0}$   
3: end for
```

1.2 Rekursion

```
4: for  $t = 1, \dots, T$   
5:   for  $k = 1, \dots, n$   
6:      $\alpha_k(t) \leftarrow E_{k, y_t} \cdot \sum_{l=1}^n \alpha_l(t-1) P_{lk}$   
7:   end for  
8: end for
```

2. Backward-algoritmen

2.1 Slutvillkor

```
9: for  $k = 1, \dots, n$   
10:   $\beta_k(T) \leftarrow 1$   
11: end for
```

2.2 Rekursion

```
12: for  $t = T-1, \dots, 0$   
13:   for  $k = 1, \dots, n$   
14:      $\beta_k(t) \leftarrow \sum_{l=1}^n P_{kl} E_{l, y_{t+1}} \beta_l(t+1)$   
15:   end for  
16: end for
```

3.1 Forward-backward-algoritmen

```
17: for  $t = 0, \dots, T$   
18:   for  $k = 1, \dots, n$   
19:      $\mathbb{P}(X_t = x_k \mid \mathbf{y}) \leftarrow \frac{\alpha_k(t) \beta_k(t)}{\sum_{j=1}^n \alpha_j(t)}$   
20:   end for  
21: end for
```

Forward-backward-algoritmen

I forward-backward-delen av algoritm 1 beräknas produkten av forward-variablen och backward-variabeln

$$\begin{aligned}\alpha_k(t)\beta_k(t) &= \mathbb{P}(X_t = x_k, O_t = y_t, \dots, O_0 = y_0) \mathbb{P}(O_{t+1} = y_{t+1}, \dots, O_T = y_T \mid X_t = x_k) \\ &= \mathbb{P}(X_t = x_k, O_0 = y_0, \dots, O_T = y_T)\end{aligned}$$

vilket är sannolikheten som algoritmen returnerar i täljaren [11][10]. För att få den betingade sannolikheten $\mathbb{P}(X_t = x_k \mid \mathbf{y})$ divideras med summan $\sum_{j=1}^n \alpha_j(T) = \mathbb{P}(\mathbf{y})$. Sannolikheterna ger en a posteriori fördelning i form av en matris för alla tider och tillstånd.

2.2.2 Viterbi-algoritmen

Viterbi-algoritmen förutsätter samma parametrar som redan har beskrivits för forward-backward-algoritmen [13]. Den kan beskrivas i tre delar: initialisering, rekursion och tillbakaspårning.

Algoritm 2 Viterbi-algoritmen [13]

Förutsättning: Observationssekvens $\mathbf{y}$, startfördelning $\boldsymbol{\pi}$, övergångsmatris P samt emissionsmatris E .

Resultat: Den mest sannolika tillståndsssekvensen $X^* = (X_1^*, \dots, X_T^*)$.

Initialisering

- 1: **for** $k = 1, \dots, n$
- 2: $\delta_k(0) \leftarrow \boldsymbol{\pi} \cdot E_{k,y_0}$
- 3: $\psi_0(k) \leftarrow 0$
- 4: **end for**

Rekursion

- 5: **for** $t = 1, \dots, T$
- 6: **for** $k = 1, \dots, n$
- 7: $\delta_k(t) \leftarrow \max_{l=1, \dots, n} [\delta_l(t-1) P_{lk}] \cdot E_{k,y_t}$
- 8: $\psi_t(k) \leftarrow \arg \max_{l=1, \dots, n} [\delta_l(t-1) P_{lk}]$
- 9: **end for**
- 10: **end for**

Sista tillstånd

- 11: $X_T^* \leftarrow \arg \max_{k=1, \dots, n} \delta_k(T)$

Tillbakaspårning

- 12: **for** $t = T, \dots, 1$
 - 13: $X_{t-1}^* \leftarrow \psi_t(X_t^*)$
 - 14: **end for**
-

Algoritmen använder sig främst av två variabler, $\delta_k(t)$ och $\psi_t(k)$ [10][13]. Den första betecknar sannolikheten för den mest sannolika tillståndsssekvens som slutar i tillstånd x_k vid tidpunkt t , givet observationerna fram till och med tidpunkt t . Den andra, $\psi_t(k)$, anger det föregående tillstånd som ingår i den mest sannolika vägen till tillstånd x_k vid tidpunkt t . Den mest sannolika sekvensen av dolda tillstånd givet observationssekvensen $\mathbf{y}$ betecknas $X^* = (X_1^*, \dots, X_T^*)$.

I initialiseringen finns inget föregående tillstånd vilket ger att $\psi_0(k) = 0$ för alla k , och sannolikheten ges då genom produkten av startfördelningen och emissionssannolikheten, det vill säga $\delta_k(0) = \boldsymbol{\pi} E_{k,y_0}$ [13].

I rekursionen beräknas $\delta_k(t) = \max_{l=1, \dots, n} [\delta_l(t-1) P_{lk}] \cdot E_{k,y_t}$ [13]. Uttrycket $\delta_l(t-1) \cdot P_{lk}$ representerar sannolikheten för den mest sannolika tillståndsväg som leder till tillståndet x_l , följt av en

övergång till x_k . Det föregående tillstånd x_l som maximerar detta uttryck lagras genom variabeln $\psi_t(k)$.

I slutsteget bestäms sedan det sista tillståndet i den mest sannolika sekvensen X_T^* genom att välja det tillstånd x_k som maximerar $\delta_k(T)$ [13]. Till sist rekonstrueras den optimala tillståndssekvensen X^* genom tillbakaspårning med hjälp av variabeln $\psi_t(k)$.

2.3 Modellformulering

Vi betraktar en sjukhusavdelning med N vårdplatser. Varje vårdplats rymmer en patient som kan vara koloniserad (smittad) eller okoloniserad (ej smittad). Sjukhusavdelningens tillstånd specificeras av vilka patienter som är koloniserade respektive okoloniserade. Detta representeras av binära vektorer $\mathbf{x} \in \{0, 1\}^N =: \mathcal{S}$, där $\mathbf{x}_j = (x_j^{(1)}, x_j^{(2)}, \dots, x_j^{(N)})$ och $|\mathcal{S}| = 2^N$. Här indikerar $x_j^{(i)} = 1$ att patient i är koloniserad och $x_j^{(i)} = 0$ indikerar att patient i är okoloniserad för $i = 1, \dots, N$. Denna representation utgör utgångspunkten för den dolda Markovmodellen som formuleras i de två följande avsnitten.

2.3.1 Markovkedjan

Varje steg i Markovkedjan antas ske på ett sådant sätt att endast en *händelse* kan inträffa på ett tidsteg. En händelse definieras som att exakt en patient blir koloniserad eller att exakt en patient blir utskriven och sedan skrivs direkt en ny patient in. Nyinskrivna patienter antas vara okoloniserade. För att karakterisera en händelse nyttjas Hamming-avståndet som definieras $d_H(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N \mathbf{1}(x^{(i)} \neq y^{(i)})$, vilket är antalet positioner där vektorerna skiljer sig åt [14]. Detta ger att en händelse definieras av en övergång från ett tillstånd $\mathbf{x} \in \mathcal{S}$ till ett tillstånd $\mathbf{y} \in \mathcal{S}$ sådant att $d_H(\mathbf{x}, \mathbf{y}) \leq 1$.

För att beskriva i vilken riktning tillstånden har förändrats introduceras Hamming-vikten, $w_H : \{0, 1\}^N \rightarrow \mathbb{N}$ definierad som $w_H(\mathbf{x}) = d_H(\mathbf{x}, \mathbf{0})$ [14]. För övergången från $\mathbf{x}$ till $\mathbf{y}$ där $d_H(\mathbf{x}, \mathbf{y}) = 1$ får vi för något $i = 1, \dots, N$ att $x^{(i)} = 1$ och $y^{(i)} = 0$ om $w_H(\mathbf{y}) - w_H(\mathbf{x}) = -1$, dvs. en utskrivning. På samma sätt får vi att $x^{(i)} = 0$ och $y^{(i)} = 1$ om $w_H(\mathbf{y}) - w_H(\mathbf{x}) = 1$, dvs. en kolonisering.

Antag att sjukhusavdelningen befinner sig i tillståndet $\mathbf{x} \in \mathcal{S}$. Låt antalet koloniserade patienter i detta tillstånd betecknas K och antalet mottagliga/okoloniserade patienter betecknas $m_{\mathbf{x}}$, då får vi att antalet okoloniserade är $m_{\mathbf{x}} = N - K$. För det givna tillståndet finns tre kvalitativt olika övergångar till ett tillstånd $\mathbf{y}$ som faller under vår definition av en *händelse*. Dessa är som följer:

1. Utskrivning av en okoloniserad och en direkt inskrivning av en okoloniserad patient. Vilket motsvaras av att $d_H(\mathbf{x}, \mathbf{y}) = 0$.
2. En kolonisering av *en* patient. Vilket motsvarar $d_H(\mathbf{x}, \mathbf{y}) = 1$ och $w_H(\mathbf{y}) - w_H(\mathbf{x}) = 1$
3. En utskrivning av en koloniserad patient. Vilket motsvarar $d_H(\mathbf{x}, \mathbf{y}) = 1$ och $w_H(\mathbf{y}) - w_H(\mathbf{x}) = -1$.

Tiden för att en koloniserad patient i smittar en okoloniserad patient j representeras av slumpvariabeln Z_{ij} . Slumpvariablerna antas vara oberoende och identiskt fördelade, och är exponentialfördelade med parametern λ

$$Z_{ij} \sim \exp(\lambda), \quad \forall i, j, \quad i \neq j. \quad (5)$$

Tiden för att det sker en utskrivning av patient i från sjukhusavdelningen representeras av slumpvariabeln Y_i . Slumpvariablerna antas vara oberoende och identiskt fördelade, och är exponentialfördelade med parametern μ

$$Y_i \sim \exp(\mu) \quad \forall i. \quad (6)$$

Nästa händelse givet tillståndet $\mathbf{x}$ bestäms av minimum av alla dessa oberoende exponentialfördelade tider. Följande resultat används för att beräkna övergångssannolikheterna [15].

Proposition 2.1. Låt $X_1, \dots, X_n$ vara oberoende slumpvariabler sådana att $X_i \sim \exp(\lambda_i)$ och $\lambda_i > 0$ för $i = 1, \dots, n$. Definiera $Y = \min_{1 \leq i \leq n} X_i$ och $\Lambda = \sum_{i=1}^n \lambda_i$. Då gäller att:

$$\mathbb{P}(Y = X_k) = \frac{\lambda_k}{\Lambda}$$

och $Y \sim \exp(\Lambda)$.

Sannolikheten att en specifik utskrivning Y_l inträffar först ges därför av

$$\begin{aligned} p_u(\mathbf{x}) &= \mathbb{P}(Y_l = \min(\{Z_{ij} : i = 1, \dots, K, j = 1, \dots, m\} \cup \{Y_i : i = 1, \dots, N\})) \\ &= \frac{\mu}{K(N - K)\lambda + N\mu}. \end{aligned} \quad (7)$$

På motsvarande sätt erhålls sannolikheten att en kolonisering inträffar först genom

$$\begin{aligned} p_s(\mathbf{x}) &= \mathbb{P}(Z_{ij} = \min(\{Z_{ij} : i = 1, \dots, K, j = 1, \dots, m\} \cup \{Y_i : i = 1, \dots, N\})) \\ &= \frac{K\lambda}{K(N - K)\lambda + N\mu}. \end{aligned} \quad (8)$$

Sammanfattningsvis definieras Markovkedjan för den dolda smittspridningsprocessen av övergångssannolikheterna:

$$P_{ij} = \begin{cases} 0 & \text{om } d_H(\mathbf{x}_i, \mathbf{x}_j) > 1 \\ p_s(\mathbf{x}_i) & \text{om } (d_H(\mathbf{x}_i, \mathbf{x}_j) = 1) \wedge (w_H(\mathbf{x}_j) - w_H(\mathbf{x}_i) = 1) \\ p_u(\mathbf{x}_i) & \text{om } (d_H(\mathbf{x}_i, \mathbf{x}_j) = 1) \wedge (w_H(\mathbf{x}_j) - w_H(\mathbf{x}_i) = -1) \\ m_{\mathbf{x}_i} p_u(\mathbf{x}_i) & \text{om } d_H(\mathbf{x}_i, \mathbf{x}_j) = 0 \end{cases} \quad (9)$$

för $i = 1, \dots, 2^N - 1$ och $j = 0, \dots, 2^N - 1$. Notera också att alla rader i matrisen har följande radsumma

$$\sum_{j=0}^{2^N-1} P_{ij} = \sum_{l=1}^m p_s(\mathbf{x}_i) + \sum_{r=1}^N p_u(\mathbf{x}_i) = 1 \quad \forall i = 1, \dots, 2^N - 1 \quad (10)$$

och att T alltså är en stokastisk matris.

Det bör noteras att tillståndet $\mathbf{x} = (0, 0, \dots, 0)$ utgör ett specialfall. I tillståndet $\mathbf{x} = (0, 0, \dots, 0)$ finns inga koloniserade och därför kan ingen kolonisering ske. Den enda möjliga händelsen är en utskrivning följt av en inskrivning av en okoloniserad patient, vilket innebär att detta är ett absorberande tillstånd. Det vill säga $P_{00} = 1$ och $P_{0j} = 0$ för alla $j = 1, \dots, 2^N$ [15].

2.3.2 Emissionsmatris

I observationsprocessen antas en observation vara sådan att varje patient testas oberoende med ett perfekt test med sannolikhet p . Utfallsrummet för observationer är då $\Omega = \{0, 1, \emptyset\}^N$, där $\mathbf{o}_j = (o_j^{(1)}, o_j^{(2)}, \dots, o_j^{(N)})$. Här indikerar $o^{(i)} = 0$ att patient i är okoloniserad, $o^{(i)} = 1$ indikerar att patient i är koloniserad och $o^{(i)} = \emptyset$ indikerar att patient i ej är testad.

Låt $X_t^{(i)}$ och $O_t^{(i)}$ beteckna tillståndet respektive observationen för patient i vid tidpunkt t . För varje patient $i = 1, \dots, N$ gäller då att

$$\mathbb{P}\left(O_t^{(i)} = o^{(i)} \mid X_t^{(i)} = x^{(i)}\right) = \begin{cases} p, & \text{om } o^{(i)} = x^{(i)}, \\ 1 - p, & \text{om } o^{(i)} = \emptyset, \\ 0, & \text{annars,} \end{cases} \quad (11)$$

vilket följer av att testen är perfekta och att varje patient testas med sannolikhet p . Eftersom testningen sker oberoende mellan patienter, givet det dolda tillståndet, faktoreriserar emissions sannolikheten som

$$E_{ij} = \mathbb{P}(O_t = \mathbf{o}_i \mid X_t = \mathbf{x}_j) = \prod_{l=1}^N \mathbb{P}\left(O_t^{(l)} = o_i^{(l)} \mid X_t^{(l)} = x_j^{(l)}\right) \quad (12)$$

2.3.3 Väntevärde av a posteriorifördelningen

För att kunna jämföra resultaten av forward-backward-algoritmen 1 för olika realisationer av modellen introduceras väntevärdet av den resulterade a posteriori fördelningen $\mathbb{P}(X_j \mid O_1 = o_1, \dots, O_t = o_t)$ enligt

$$\mathbb{E}[X_j \mid O_1, \dots, O_t] = \sum_i x_i \cdot P(X_j = x_i \mid O_1 = o_1, \dots, O_t = o_t) \quad \forall j = 1, \dots, t. \quad (13)$$

Eftersom tillstånden x_i är binära vektorer ger väntevärdet en vektor med komponenter i intervallet $[0, 1]$.

3 Metod

Detta avsnitt beskriver hur studien genomförts och vilka metoder som använts för att undersöka inferensalgoritmernas förmåga att återskapa den dolda smittspridningsprocessen. All data genererades genom simulering av den dolda Markovmodell som definieras i avsnitt 2.3. För varje simulering genererades först en realisation av den dolda tillståndsprocessen, vilken därefter användes för att generera en motsvarande observationssekvens enligt emissionsmodellen i avsnitt 2.3.2. De simulerade observationerna användes därefter som indata i forward-backward-algoritmen respektive Viterbi-algoritmen.

På grund av det exponentiellt växande tillståndsrummet användes glesa matriser vid lagring och beräkning av övergångsmatrisen och för inferensalgoritmerna. Viterbi-algoritmen tidsoptimerades dessutom med hjälp av Python-biblioteket `numba` för att möjliggöra simuleringar för större tillståndsrum.

För att utvärdera algoritmernas prestanda definierades noggrannhet för Viterbi-algoritmen och forward-backward-algoritmen. För Viterbi-algoritmen användes två mått på noggrannhet: den tillståndsvisa noggrannheten och den komponentvisa noggrannheten. Den tillståndsvisa noggrannheten beräknades genom att det skattade tillståndet jämfördes med det dolda tillståndet, där ett tillstånd betraktades som korrekt endast om samtliga komponenter överensstämde. Den komponentvisa noggrannheten beräknades genom att varje komponent i det skattade tillståndet jämfördes med motsvarande komponent i det dolda tillståndet. Forward-backward-algoritmen gav en a posteriori sannolikhetsfördelning över de dolda tillstånden, vilken omvandlades till en väntevärdessekvens enligt teoriavsnitt 2.3.3. Noggrannheten beräknades genom komponentvis jämförelse mellan den skattade sekvensen och den dolda kedjan, där varje komponent bidrog med ett värde mellan 0 och 1.

3.1 Testningsgradens påverkan

Analysen av hur testningsgraden påverkade resultaten genomfördes genom att variera testsannolikheten p i intervallet $[0, 1]$. Emissionsmatrisen konstruerades för varje värde på p enligt teoriavsnitt 2.3.2. Parametrarna för smittotiden och utskrivningstiden, i enlighet med teoriavsnitt 2.3.1, hölls konstanta på $\lambda^{-1} = 5,3$ respektive $\mu^{-1} = 1$. För varje värde på p genomfördes 1000 simuleringar, där noggrannheten för respektive algoritm beräknades. Därefter beräknades medelvärdet av resultaten för varje värde på p .

3.2 Infektivitetens påverkan

För att undersöka infektivitetens påverkan varierades förhållandet mellan λ och μ , medan testningssannolikheten hölls konstant på $p = 0,4$. Först studerades infektivitetens påverkan på den dolda kedjan och sedan Viterbi-algoritmens noggrannhet för olika förhållanden. Eftersom det var förhållandet mellan tiderna som var av intresse fixerades $\mu^{-1} = 1$ medan λ^{-1} varierades logaritmiskt i intervallet $[10^{-2}, 10^3]$. För varje värde på λ^{-1} genomfördes 1000 simuleringar med maximalt 100 tidssteg.

Eftersom Markovmodellen inte bara har ett absorberande tillstånd $(0, \dots, 0)$ utan även ett kvasiabsorberande tillstånd $(1, \dots, 1)$ kunde inte teorin om förväntad tid till absorberande tillstånd användas. Full kolonisation definierades som kvasiabsorberande för att när infektiviteten var hög i förhållande till utskrivningen *fastnade* kedjan i en cykel vid $(1, \dots, 1)$ där den gick ett steg ifrån och ett tillbaka. Därför konstruerades ett test som undersökte hur snabbt kedjan *fastnade* i något av dessa tillstånd. Syftet med testet var att identifiera vilket förhållande på parametrarna λ och μ som gav den längsta kedjan innan den *fastnat*.

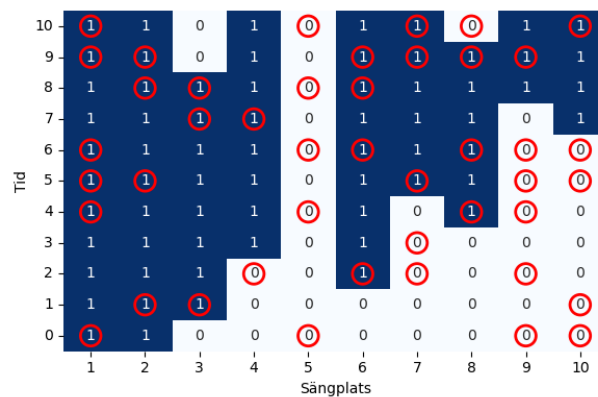
För varje simulering bestämdes först det tidssteg t_{steg} där kedjan först *fastnade* och omvandlades därefter till ett mått på hur snabbt det inträffade. För att samtidigt skilja mellan de två tillstånden gavs det kvasiabsorberande tillståndet ett positivt värde och det absorberande ett negativt värde. För det kvasiabsorberade tillståndet definierades genomsnittstiden till att *fastna* som $(1 - \frac{t_{steg}}{100})$, medan för det absorberande tillståndet definierades som $-(1 - \frac{t_{steg}}{100})$. Det kvasiabsorberade tillståndet definierades ha *fastnat* i t_{steg} om kedjan nådde tillståndet $(1, \dots, 1)$ vid t_{steg} och därefter lämnade och återvände till tillståndet tre gånger i följd.

Infektivitetens påverkan på algoritmernas förmåga att estimerar den dolda kedjan analyserades på motsvarande sätt som ovan. Utskrivningstiden hölls konstant, $\mu^{-1} = 1$, medan λ^{-1} varierades. För varje värde på λ^{-1} genomfördes 1000 simuleringar där noggrannheten beräknades enligt ovan, varefter medelvärdet togs över simuleringarna.

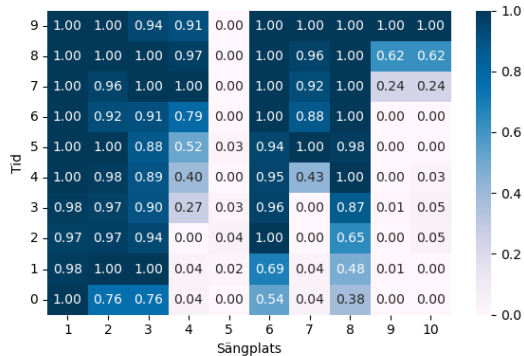
4 Resultat

I detta avsnitt presenteras resultaten från simuleringsstudien. Först ges en illustration av algoritmernas skattning för ett enskilt simulerat förlopp, följt av hur testningsgraden påverkar noggrannheten för respektive algoritm. Avslutningsvis presenteras hur infektiviteten påverkar både den dolda kedjans beteende och Viterbi-algorithmens förmåga att skatta tillståndsssekvensen.

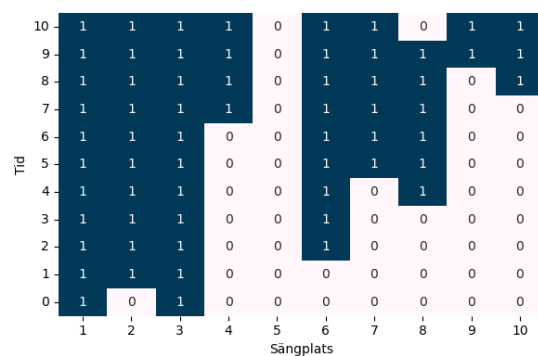
För att ge en kvalitativ förståelse av algoritmernas skattningar presenteras ett simulerat händelseförlopp för samtliga sängplatser. I figur 2 presenteras forward-backward-algorithmens och Viterbi-algorithmens estimeringar av hela händelseförloppet i jämförelse med det dolda händelseförloppet. I figur 2a presenteras den dolda sekvensen av händelseförloppet med tillstånd för alla individer över tid, där de röda cirkelarna indikerar att en patient testades. Forward-backward-algorithmens skattning av händelseutvecklingen, där komponenternas väntevärden erhålls genom viktning av a posteriori fördelningen, presenteras i figur 2b, och Viterbi-algorithmens skattning presenteras i figur 2c.



(a) Den dolda sekvensen av händelseförloppet. Värdet 1 illustrerar att patienten i sängplatsen är koloniserad och värdet 0 betyder att den är okoloniserad. De röda cirkelarna markerar observationerna.



(b) Forward-backward-algorithmens estimering av det dolda händelseförloppet.



(c) Viterbi-algorithmens estimering av det dolda händelseförloppet.

Figur 2: Jämförelse av dold och estimerad tillståndsssekvens.

I figur 2b och figur 2c visas att Viterbi-algoritmen och forward-backward-algoritmen ger liknande estimeringar. I starttillståndet uppskattar Viterbi-algoritmen inkorrekt gällande sängplats 2 och 3 angående smittotillståndet. Forward-backward-algoritmen anger istället en osäkerhet, med en lika stor sannolikhet för att dessa patienter är sjuka. Givet observationerna är det tydligt att sannolikheten är lika stor för båda om startfördelningen är samma. Forward-backward-algorithmens osäkerhet minskar betydligt desto närmare en observation den kommer. Ett exempel på detta är

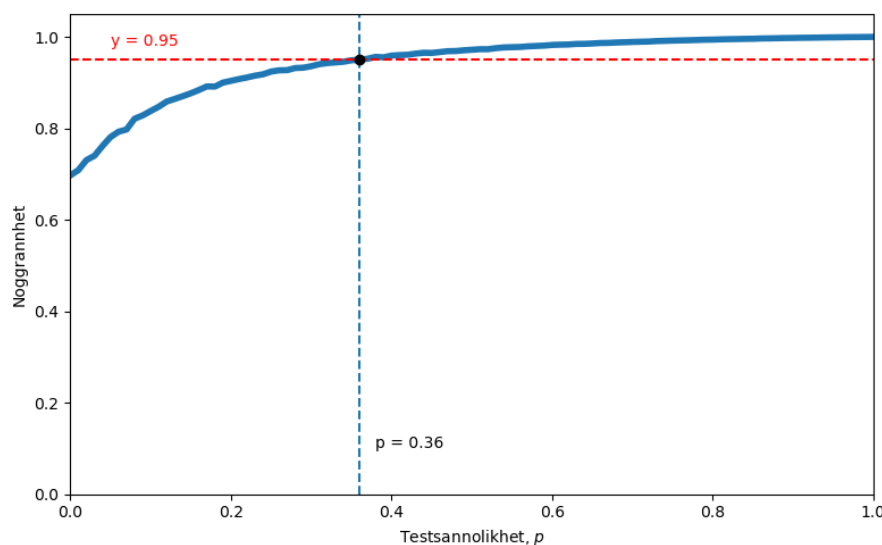
sängplats 4 som vid tidpunkt 3 är 0,27 och ökar successivt tills nästa observation som sker vid tidpunkt 7. I jämförelse tenderar Viterbi-algoritmen att stanna på okoloniserad tills en observation görs att patienten är koloniserad.

4.1 Testningsgradens påverkan

Följande delavsnitt visar hur testningsgraden p påverkar Viterbi-algoritmens skattning av de dolda tillstånden och forward-backward-algoritmens väntevärdes skattning.

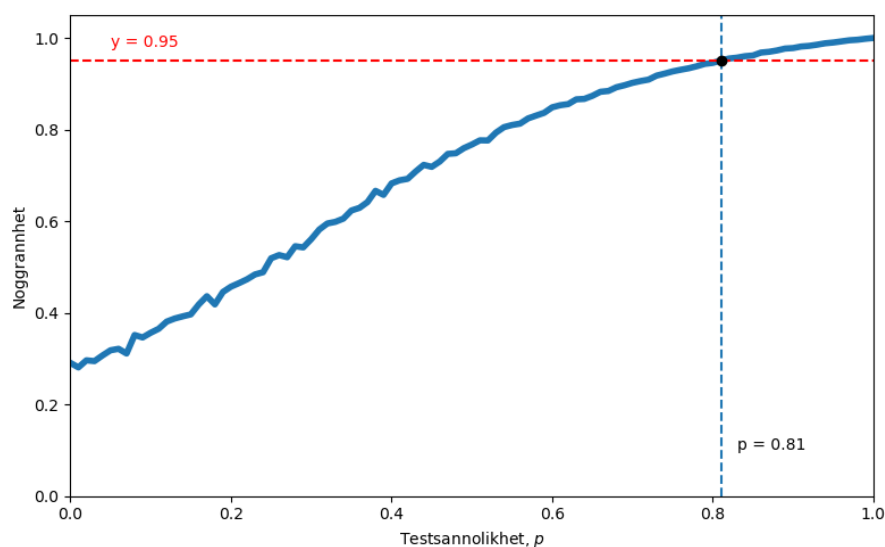
4.1.1 Viterbi-algoritmen

Figur 3 visar hur Viterbi-algoritmens komponentvisa noggrannhet påverkas av testningsgraden p . Andelen korrekt identifierade komponenter ökar snabbt redan vid låga värden på p , där noggrannheten överstiger 80%. Därefter fortsätter noggrannheten att öka, men i avtagande takt, och närmar sig 1 då $p \rightarrow 1$. En komponentvis noggrannhet på 95% uppnås vid $p \approx 0,36$.



Figur 3: Den komponentvisa noggrannheten som funktion av testningsgraden p för Viterbi-algoritmen.

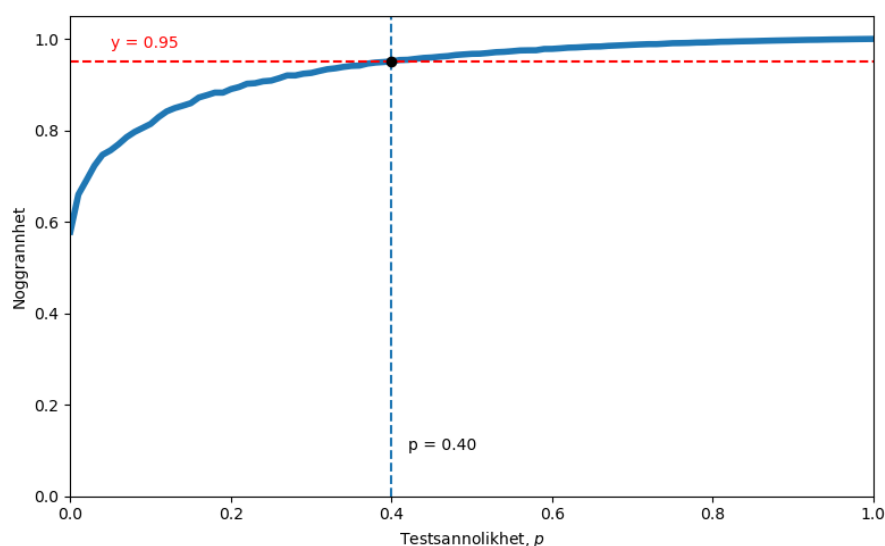
I figur 4 visas hur Viterbi-algoritmens tillståndsvisa noggrannhet påverkas av testningsgraden p . Noggrannheten ökar monotont mot 1 när $p \rightarrow 1$ och likt den komponentvisa noggrannheten så avtar ökningen när p ökar. Den tillståndsvisa kurvan förefaller ha en brantare ökning över intervallet än den komponentvisa. Vid $p = 0$ är den tillståndsvisa noggrannheten lägre än den komponentvisa. Jämfört med den komponentvisa analysen krävs en högre testningsgrad för att uppnå en tillståndsvis noggrannhet på 95 %, vilket sker när $p = 0,81$.



Figur 4: Den tillståndsvisa noggrannheten som funktion av testningsgraden p för Viterbi-algoritmen.

4.1.2 Forward-backward-algoritmen

I figur 5 visas hur forward-backward-algorithmens komponentvisa noggrannhet påverkas av testningsgraden p . Vid $p = 0$ är den komponentvisa noggrannheten runt 58% och går mot 1 då $p \rightarrow 1$, där ökningen avtar med växande p . I jämförelse med Viterbi-algorithmens komponentvisa noggrannhet når forward-backward-algoritmen en komponentvis noggrannhet på 95% vid en högre testningsgrad, när $p = 0,40$.



Figur 5: Noggrannheten för forward-backward-algoritmen som funktion av testningsgraden p .

En sammanställning av testningsgraderna som motsvarar minst 95% korrekta resultat i genomsnitt, gällande de studerade algoritmerna, noteras i tabell 1.

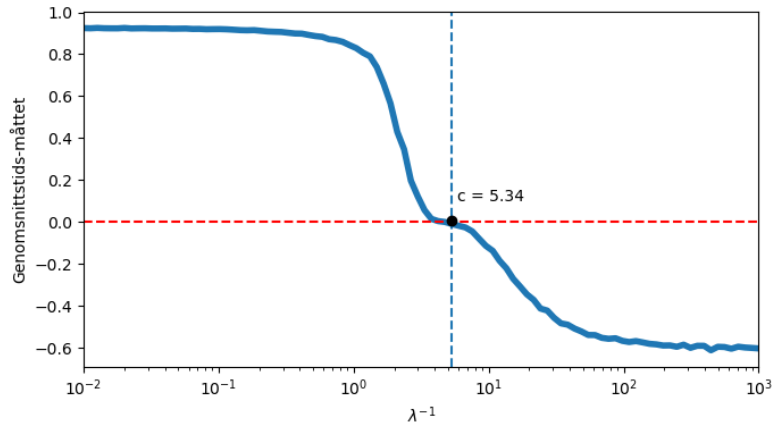
Tabell 1: Resultat för algoritmernas noggrannhet vid 95 % testnoggrannhet.

Algoritm	Testningsgrad
Viterbi _{komponentvis}	0,36
Viterbi _{tillståndsvi}	0,81
Forward-backward _{komponentvis}	0,40

4.2 Infektivitetens påverkan

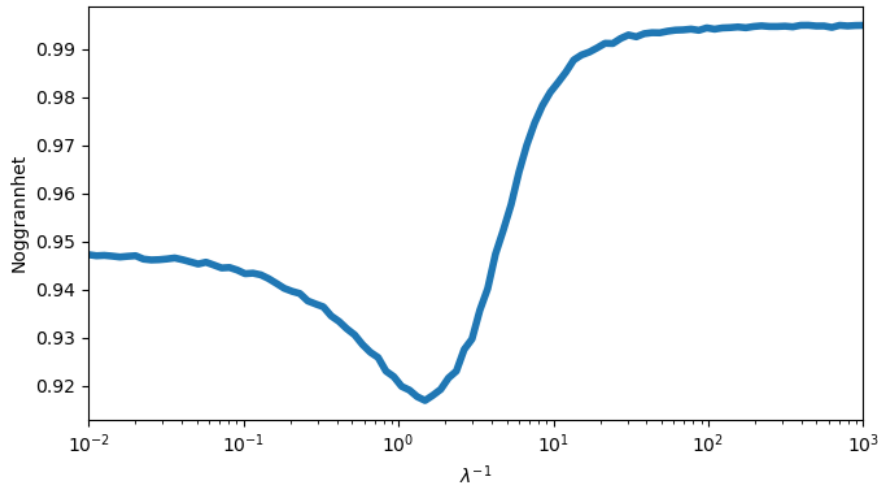
För att analysera infektivitetens påverkan presenteras först hur den dolda Markovkedjans kvalitativa beteende förändras, mätt med genomsnittstiden till att *fastna*. Därefter presenteras hur samma variation i infektivitet påverkar Viterbi-algoritmens komponentvisa noggrannhet.

I figur 6 presenteras resultatet av måttet av genomsnittstiden för fixerad parameter $\mu^{-1} = 1$, och varierande λ^{-1} . När väntevärdet för smittotiden är mindre än väntevärdet för utskrivningstiden, $\lambda^{-1} < \mu^{-1}$, är måttet ungefär lika med 1 vilket indikerar att Markovkedjan kommer nå tillståndet $(1, 1, \dots, 1)$ och *fastna* tidigt i realisationen. I motsatt fall, $\lambda^{-1} \gg \mu^{-1}$, sjunker måttet till ungefär $-0,75$ och kedjan når istället det absorberande tillståndet tidigt i realisationen. I det mellanliggande området då, $\lambda^{-1} \approx \frac{c}{\mu}$, har vi att måttet är ungefär lika med 0, vilket indikerar att kedjan i genomsnitt når något av de två tillstånden sent i realisationen, eller inte alls.



Figur 6: Mått av genomsnittstid för att *fastna* som funktion av λ^{-1} för fixerat $\mu = 1$.

I figur 7 presenteras hur andelen korrekta komponenter skattade av Viterbi-algoritmen förändras när λ^{-1} varieras och $\mu^{-1} = 1$. När väntevärdet för smittotiden är stort i förhållande till väntevärdet för utskrivningstiden, $\lambda^{-1} \gg \mu^{-1}$, är noggrannheten hög. Detsamma gäller när väntevärdet för smittotiden är litet i förhållande till väntevärdet för utskrivningstiden, $\lambda^{-1} \ll \mu^{-1}$, även om noggrannheten i detta fall är något lägre. I det mellanliggande området där $\lambda \approx \mu$ uppnås den lägsta noggrannheten.



Figur 7: Medelvärde av andel korrekt estimerade komponenter av Viterbi-algoritmen som funktion av smittotiden λ^{-1} för fixerad utskrivningstid $\mu = 1$.

5 Diskussion

I detta avsnitt diskuteras resultaten i förhållande till studiens två frågeställningar: testningsgradens och infektivitetens påverkan på algoritmernas noggrannhet. Därefter behandlas modellens begränsningar samt möjliga utvidgningar för framtida studier.

Resultaten visar att både Viterbi-algoritmen och forward-backward-algoritmen uppnår bättre noggrannhet vid högre testningsgrad, vilket är förväntat eftersom mer information om det dolda tillståndet naturligtvis ger en bättre skattning. Det är också betydligt enklare för Viterbi-algoritmen att korrekt skatta enskilda komponenter i tillståndet än hela tillståndet, eftersom en felklassificerad komponent endast påverkar den komponentvisa noggrannheten i begränsad omfattning, till skillnad från den tillståndsvisa noggrannheten där samtliga komponenter måste klassificeras korrekt samtidigt. Slutligen noteras för testsannolikheten att tröskelvärdena baseras på medelvärden över ett antal simuleringar och därmed inte garanterar att varje enskild sekvens uppnår samma noggrannhet. Dessutom bygger modellen på diskreta händelsesteg snarare än verklig tid, vilket försvårar en realistisk tolkning av tröskelvärdena i praktiska tillämpningar.

Infektivitetens påverkan kan förstås genom att koppla genomsnittstiden till att *fastna* till Viterbi-algorithmens komponentvisa noggrannhet, där det förra beskriver den dolda kedjans beteende och det senare hur väl algoritmen lyckas återskapa det. Resultaten tyder på att noggrannheten är hög när kedjan snabbt rör sig mot ett tydligt läge, vilket sker i två fall. När väntevärdet för smittotiden är stort i förhållande till väntevärdet för utskrivningstiden absorberas kedjan tidigt i det smittfria tillståndet. Därefter förändras tillståndet inte, och algoritmen behöver i praktiken endast estimerar en konstant sekvens, vilket förklarar varför noggrannheten är som högst i detta område. När väntevärdet för smittotiden istället är litet besöker kedjan tidigt och frekvent tillståndet där samtliga patienter är koloniserade. Den variation som återstår att estimerar är den som uppstår när en koloniserad patient skrivs ut och en ny snabbt smittas, en liten men återkommande förändring som ändå är enklare att skatta än den mer oregelbundna dynamiken i det mellanliggande området.

Modellen i sin nuvarande form bygger på ett antal förenklande antaganden som är värda att diskutera. Ett sådant antagande är användningen av definitionen av händelser som tidssteg, vilket gjordes för att erhålla glesa matriser. Resultatet av detta är en svårtolkad modell. En naturlig utvidgning vore att använda dagar som tidssteg, vilket dock kräver både större datorkapacitet och mer detaljerad data om smittotider. En annan begränsning är att väntevärdet för smittotiden λ antogs vara detsamma för alla patienter, trots att en av fördelarna med den individbaserade modellen är möjligheten att modellera heterogenitet. En möjlig utvidgning skulle vara att införa

N vårdplatsspecifika smittotids-parametrar $\lambda_1, \dots, \lambda_N$ för att modellera heterogenitet. Även om det bör noteras att detta snarare modellerar heterogenitet hos vårdplatserna än hos de enskilda patienterna, vilket inses genom att olika patienter kan skrivas in och ut från sängplatsen. Modellen utnyttjar heller inte all information som rimligen bör finnas tillgänglig för en sjukhusavdelning, framförallt vilka patienter som blir utskrivna. Detta skulle potentiellt kunna användas för att förbättra skattningarna. Slutligen noteras två antaganden som orealistiska: att ingen patient som skrivs in kan vara koloniserad, samt att en patient ej tillfrisknar utan enbart kan bli utskriven.

5.1 Samhälleliga och etiska aspekter

I detta arbete används enbart simulerad data för att genomföra inferens. De metoder som används är dock avsedda att kunna tillämpas på verkliga data. I ett sådant sammanhang uppstår flera etiska utmaningar kopplade till insamling och hantering av data. Den typ av data som krävs för modellen innefattar information om individers hälsotillstånd, vilket klassificeras som känsliga personuppgifter [16]. Insamling och lagring av sådan data ställer därför höga krav på integritetsskydd, exempelvis genom anonymisering och säker hantering. Samtidigt finns en risk för återidentifiering, särskilt om data kombineras med andra informationskällor. Vidare uppstår frågor kring samtycke, då individer har rätt att informeras om hur deras data används och i vilket syfte den samlas in [16]. Det är också viktigt att beakta proportionalitetsprincipen, vilket i detta sammanhang innebär att endast den data som är nödvändig för analysens syfte bör samlas in.

Man bör beakta att modellen som formuleras i denna studie utgör en förenklad representation av verkligheten och därför inte återger alla aspekter av den faktiska smittspridningsprocessen. Resultat från algoritmerna ger i allmänhet endast probabilistiska skattningar med inneboende osäkerhet. Om en sådan modell används i en verklig vårdssituation och resultaten tolkas som exakta beskrivningar finns en risk att felaktiga åtgärder vidtas, exempelvis att en tidigare patient och alla personer denne har varit i kontakt med sätts i isolation när modellen felaktigt visat att personen var koloniserad.

5.2 Slutsats

Slutligen visar studien att relationen mellan λ och μ , samt testningsgraden har stor påverkan på algoritmernas noggrannhet och den dolda Markovkedjan. Noggrannheten påverkas både av testningsgraden samt om klassificering sker komponentvis eller tillståndsvis. Viterbi-algorithmens komponentvisa noggrannhet påverkas tydligt av relationen mellan λ och μ . Det framgår att modellen är väl anpassad för samtliga inferensalgoritmer, särskilt vid komponentvis estimering, eftersom tillståndssekvenser kan rekonstrueras med hög noggrannhet vid relativt låg testningsgrad. Modellen vilar på ett antal förenklande antaganden, vilka utgör naturliga utgångspunkter för framtida utvidgningar av studien för att förbättra realismen och möjligen tillämpbarheten.

Referenser

1. Omran AR. The Epidemiologic Transition: A Theory of the Epidemiology of Population Change. *Milbank Mem Fund Q.* 1971; 49. DOI: 10.1111/j.1468-0009.2005.00398.x
2. Vos T, Lim SS, Abbafati C, Abbas KM, Abbasi M, Abbasifard M m. fl. Global burden of 369 diseases and injuries in 204 countries and territories, 1990–2019: A systematic analysis for the Global Burden of Disease Study 2019. *Lancet.* 2020; 396:1024–222. DOI: 10.1016/S0140-6736(20)30925-9
3. Naghavi M, Vollset SE, Ikuta KS, Swetschinski LR, Gray AP, Wool EE m. fl. Global burden of bacterial antimicrobial resistance 1990–2021: A systematic analysis with forecasts to 2050. *Lancet.* 2024; 404:1199–226. DOI: 10.1016/S0140-6736(24)01867-1
4. Rokib SI, Hossain S, Malek MB, Ruhi FA och Mallick UK. Mathematical Analysis on Transmission of Nosocomial Infection Applying Combination Therapy. *J Appl Math* 2025; 2025. DOI: 10.1155/jama/3002490
5. European Centre for Disease Prevention and Control. Point prevalence survey of healthcare-associated infections and antimicrobial use in European acute care hospitals. Stockholm: ECDC, 2024. DOI: 10.2900/88011
6. Socialstyrelsen. Vårdrelaterade infektioner (VRI) [Internet]. 2026. [hämtad 2026 May 2]. Available from: <https://patientsakerhet.socialstyrelsen.se/risker-och-varriskador/varriskador/vri--vardrelaterade-infektioner/>
7. Kermack WO och McKendrick AG. Contributions to the Mathematical Theory of Epidemics–I. *Bull Math Biol.* 1927; 53. DOI: 10.1007/BF02464423
8. Kleef E van, Robotham JV, Jit M, Deeny SR och Edmunds WJ. Modelling the transmission of healthcare associated infections: a systematic review. *BMC Infect Dis.* 2013; 13:294. DOI: 10.1186/1471-2334-13-294
9. Cooper B och Lipsitch M. The analysis of hospital infection data using hidden Markov models. *Biostatistics* 2004; 5:223–37. DOI: 10.1093/biostatistics/5.2.223
10. Rabiner L. A tutorial on hidden Markov models and selected applications in speech recognition. *Proc IEEE* 1989; 77:257–86. DOI: 10.1109/5.18626
11. Privault N. Discrete Stochastic Processes. 1. utg. Cham, Switzerland: Springer, 2024. DOI: 10.1007/978-3-031-65820-4
12. Grimmett GR och Stirzaker DR. Markov Chains. *Probability and Random Processes.* 3. utg. Oxford University Press, 2001 :213–304
13. Carugo O och Eisenhaber F. Data Mining Techniques for the Life Sciences. 1. utg. Totowa, NJ, USA: Humana Press, 2010 :246. DOI: 10.1007/978-1-60327-241-4
14. Betten A, Braun M, Fripertinger H, Kerber A, Kohnert A och Wassermann A. Linear Codes. *Error-correcting linear codes.* Utg. av Betten A, Braun M, Fripertinger H, Kerber A, Kohnert A och Wassermann A. Berlin: Springer Berlin Heidelberg, 2006 :3–81
15. Dobrow RP. Introduction to Stochastic Processes with R. Hoboken, NJ: Wiley, 2016
16. European Parliament and Council. General Data Protection Regulation (GDPR) (EU) 2016/679 [Internet]. Brussels, 2016. [hämtad 2026 May 5]. Available from: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

Användning av AI

Generativ AI har använts under projektets gång för att effektivisera arbetet.

I rapportskrivandet har AI använts för korrekturläsning, omformuleringar och förslag på alternativa begrepp. Texten skrevs först av författarna och matades därefter in i AI-verktyget. AI:n identifierade språkliga fel och gav förslag på nya formuleringar som granskades och implementerades manuellt. AI:n användes därför som ett stöd för att upptäcka misstag och utveckla formuleringar.

Generativ AI användes även vid programmering, både för att generera kod och för att kommentera koden. All kod granskades och anpassades så att den överensstämde med modellen. AI:n användes framförallt för att generera kod för figurer.

Generativ AI har också använts för att få en förståelse av modellen och algoritmerna. Den har använts för att ställa frågor om modellen för möjligheten för djupare förståelse.

Den har också använts för några L^AT_EX-kommandon.

A Källkod

Koden för projektet delades upp i fyra olika delar där vardera del har sin egen fil.

A.1 Generera övergångsmatris

Först genereras övergångsmatrisen med parametrarna väntevärdet för smittotiden och väntevärdet för utskrivningstiden. När denna fil kallas söker den först efter en existerande matris och om den inte finns så skapar den en ny och sparar undan denna.

```
1 import numpy as np
2 from itertools import product          #Används för att skapa våra tillstånd
3 from scipy.sparse import csr_matrix   #Paket för att skapa glesa matriser
4 import scipy.sparse as sp            #Används för att spara glesa matriser
5 from tqdm import tqdm                #Paket för att lösa progressbar
6
7 def generera_tillstand(n):
8     """
9     Genererar alla binära tillstånd (0/1-tupler) av längd n.
10
11     Detta är tillståndsrummet för Markovkedjan.
12     """
13     return list(product([0, 1], repeat=n))
14
15
16 def bygg_övergångsmatris(n, tid_smitta, tid_utskrivning):
17     """
18     Bygger en gles (CSR) övergångsmatris T för en Markovkedja där varje
19     tillstånd består av n individer med hälsostatus 0 (frisk) eller 1 (smittad).
20
21     För varje tillstånd s gäller:
22         - (0,0,...,0) : absorberande (stannar alltid)
23         - (1,1,...,1) : endast utskrivningar 1→0 möjliga
24         - blandat s   : tillåter
25
26                     * stanna kvar
27                     * exakt en bitflip (smitta 0→1 eller utskrivning
28                       ↪ 1→0)
29
30     Sannolikheterna styrs av två förväntade tider:
31         tid_smitta      → lambda_sm = 1 / tid_smitta
32         tid_utskrivning → lambda_ut = 1 / tid_utskrivning
33
34     Matrisen returneras som en scipy.sparse.csr_matrix.
35     """
36
37     # --- Omvandla tider till rates -----
38     lambda_ut = 1 / tid_utskrivning      # utskrivningsfrekvens
39     lambda_sm = 1 / tid_smitta           # smittfrekvens
40
41     # --- Generera alla tillstånd -----
42     tillstand = generera_tillstand(n)
43     N = len(tillstand)
44
45     # Dictionary för snabb O(1)-lookup: (tillståndstupel) → index i matrisen
46     idx = {s: i for i, s in enumerate(tillstand)}
47
48     # Listor för sparse-kodning (data, radindex, kolumnindex)
49     data = []
```

```

48 rows = []
49 cols = []
50
51 # --- Visa progressbar endast för stora n -----
52 # Din empiriska cutoff visade att n >= 16 är där tiden blir märkbar.
53 iterator = tqdm(range(N), desc="Bygga matris") if n >= 16 else range(N)
54
55 # =====
56 # Huvudloop: skapa matrisrad för varje tillstånd i
57 # =====
58 for i in iterator:
59     s = tillstand[i] # binärt tillstånd som tuple
60     antal_smittade = sum(s) # hur många som är 1:or
61     antal_osmittade = n - antal_smittade
62
63     # -----
64     # Fall 1: Alla friska → absorberande tillstånd
65     # -----
66     if antal_smittade == 0:
67         rows.append(i)
68         cols.append(i)
69         data.append(1.0)
70         continue
71
72     # -----
73     # Fall 2: Alla smittade → endast utskrivningar möjliga
74     # -----
75     if antal_osmittade == 0:
76         # Alla 1→0 flippar är lika sannolika
77         p = 1 / n
78         for bit in range(n):
79             if s[bit] == 1:
80                 flipped = list(s)
81                 flipped[bit] = 0
82                 j = idx[tuple(flipped)]
83
84                 rows.append(i)
85                 cols.append(j)
86                 data.append(p)
87         continue
88
89     # -----
90     # Fall 3: Blandat tillstånd
91     # Här kan både smitta (0→1), utskrivning (1→0) och stanna kvar ske.
92     # -----
93
94     # Optimering: preberäkna nämnaren i alla sannolikheter
95     denom = (
96         antal_smittade * antal_osmittade * lambda_sm
97         + n * lambda_ut
98     )
99
100     # Beräkna alla sannolikheter en gång
101     p_ut = lambda_ut / denom # utskrivning 1→0
102     p_inf = (antal_smittade * lambda_sm) / denom # smitta 0→1
103     p_stanna = antal_osmittade * p_ut # 0→0 stay-sannolikhet
104

```

```

105     # Lägg in sannolikheten att stanna i samma tillstånd
106     rows.append(i)
107     cols.append(i)
108     data.append(p_stanna)
109
110     # --- Generera alla bitflippar
111     ↪ -----
112     for bit in range(n):
113         flipped = list(s)
114         flipped[bit] ^= 1          # XOR: flippar en enda bit
115         j = idx[tuple(flipped)]    # hitta nästa tillstånds index
116
117         # 0→1 smitta
118         if s[bit] == 0:
119             data.append(p_inf)
120             # 1→0 utskrivning
121         else:
122             data.append(p_ut)
123
124     rows.append(i)
125     cols.append(j)
126
127     # =====
128     # Bygg gles övergångsmatris i CSR-format (snabb för matmul senare)
129     # =====
130     T = csr_matrix((data, (rows, cols)), shape=(N, N))
131
132     return tillstand, T
133
134 def hämta_T(n, tid_smitta=1.0, tid_utskrivning=2.0, folder="sparade_matriser"):
135     """
136     Hämtar (eller genererar) T_n med rätt ratio = tid_utskrivning/tid_smitta.
137     Filnamn beror därför på n och ratio.
138     """
139
140     # Skapa mapp om den saknas
141     if not os.path.exists(folder):
142         os.makedirs(folder, exist_ok=True)
143
144     # Beräkna ratio
145     ratio = tid_utskrivning / tid_smitta
146     ratio_str = f"{ratio:.8f}".replace(".", "_")
147
148     # Bygg filnamn
149     matrisfil = os.path.join(folder, f"T_{n}_r{ratio_str}.npz")
150     statesfil = os.path.join(folder, f"states_{n}_r{ratio_str}.npy")
151
152     # Finns filerna?
153     if os.path.exists(matrisfil) and os.path.exists(statesfil):
154         T = sp.load_npz(matrisfil)
155         raw_states = np.load(statesfil, allow_pickle=True)
156
157         # Konvertera numpy → python int + None
158         states = [
159             tuple(int(x) if x is not None else None for x in s)
160             for s in raw_states
161         ]

```

```

161         return states, T
162
163     # Annars generera och spara
164     print(f"Genererar övergångsmatris för n={n}, ratio={ratio:.4f}...")
165
166     states, T = bygg_övergångsmatris(n, tid_smitta, tid_utskrivning)
167
168     sp.save_npz(matrisfil, T)
169     np.save(statesfil, np.array(states, dtype=object))
170
171     return states, T

```

A.2 Markovkedja

En klass för Markovkedjan skapas i denna fil.

```

1  import numpy as np
2
3  class Markovkedja:
4      """
5      Klass som hanterar en diskret-tids Markovkedja definierad över ett ändligt
6      tillståndsrum av binära tupler, tillsammans med en övergångsmatris T.
7      """
8
9
10     def __init__(self, tillstand, övergångsmatris):
11         """
12         Parametrar
13         -----
14         tillstand : list[tuple]
15             Listan över tillstånd i modellen.
16             Ordningen definierar indexeringen.
17
18         övergångsmatris : csr_matrix (N x N)
19             Gles övergångsmatris där rad i listar sannolikheter till nästa
20             ↪ tillstånd.
21         """
22         self.tillstand = tillstand
23         self.T = övergångsmatris.tocsr() # säkerställ CSR-format
24         self.N = len(tillstand)
25
26         # O(1) uppslag: tillstånd → index (ersätter list.index() som är O(N))
27         self.state_to_index = {s: i for i, s in enumerate(tillstand)}
28
29         # Förberäkna transponatet T^T - extremt viktigt för snabb forward-pass
30         self.T_T = self.T.transpose().tocsr()
31
32         # Snabbare och trådsäker randomgenerator
33         self.rng = np.random.default_rng()
34
35     # -----
36     # Index <-> Tillstånd
37     # -----
38
39     def index_till_tillstand(self, index):
40         """Returnerar tillståndstupeln för ett index."""
41         return self.tillstand[index]

```

```

42
43
44 def tillstand_till_index(self, s):
45     """Returnerar index för tillstånd s i O(1)-tid."""
46     return self.state_to_index[s]
47
48
49 # -----
50 # Sampling från Markovkedjan
51 # -----
52
53 def nasta_tillstand(self, index):
54     """
55     Returnerar nästa tillståndsindex givet nuvarande index.
56     Använder CSR-matrisens interna struktur:
57
58     indptr[i] ... indptr[i+1] anger vilka kolumner som är icke-noll
59     indices[]  anger nästa tillstånd
60     data[]     anger motsvarande sannolikheter
61     """
62     start = self.T.indptr[index]
63     end = self.T.indptr[index + 1]
64
65     cols = self.T.indices[start:end] # möjliga nästa tillstånd
66     probs = self.T.data[start:end] # sannolikheter
67
68     # Returnera som Python-int, inte np.int32
69     return int(self.rng.choice(cols, p=probs))
70
71
72 # -----
73 # Simulering av Markovkedjan
74 # -----
75
76
77 def start_tills(self, p_start=0.4):
78     """
79     Skapar och lagrar en initialfördelning enligt oberoende Bernoulli(p),
80     villkorad på att vi inte startar i absorberande tillståndet 000...0.
81
82     Returnerar:
83     start_index : index för initialt tillstånd
84     start_förd  : sannolikhetsfördelning över alla tillstånd
85     """
86
87     N = self.N
88     tillstand = self.tillstand
89
90     # 1. Oberoende Bernoulli-fördelning över tillstånd
91     raw_probs = np.array([
92         (p_start ** sum(s)) * ((1 - p_start) ** (N - sum(s)))
93         for s in tillstand
94     ])
95
96     # 2. Ta bort absorberande tillstånd
97     raw_probs[0] = 0.0
98

```

```

99         if raw_probs.sum() == 0:
100             start_förd = np.ones(N)
101             start_förd[0] = 0
102             start_förd = start_förd/(N - 1)
103
104         # 3. Normalisera
105         else:
106             start_förd = raw_probs / raw_probs.sum()
107
108         # 4. Slumpa startindex enligt denna fördelning
109         start_index = self.rng.choice(N, p=start_förd)
110
111         # 5. Spara
112         self.start_förd = start_förd
113         self.start_idx = start_index
114         self.start_tillstånd = self.index_till_tillstånd(start_index)
115
116         return start_index
117
118
119     def simulera_index(self, steg, p_start = 0.4):
120         """
121         Simulerar kedjan och returnerar en lista av index.
122         """
123         start_idx = self.start_tills(p_start = p_start)
124         idx = start_idx
125         seq = [int(idx)]
126
127         for _ in range(steg):
128             idx = self.nasta_tillstånd(idx)
129             seq.append(int(idx))
130
131         return start_idx, seq
132
133
134     def simulera(self, steg, p_start = 0.4):
135         """
136         Simulerar kedjan och returnerar faktiska tillstånd (tupler).
137         """
138         start, seq = self.simulera_index(steg, p_start = p_start)
139         self.start_idx = start
140         return [self.index_till_tillstånd(i) for i in seq]
141
142

```

A.3 Dold Markovkedja

Underklassen dold Markovkedja till Markovkedja skapas här.

```

1  import numpy as np
2  from tqdm import tqdm
3  from numba import njit
4  from numba.typed import List
5
6
7  from markovkedja import Markovkedja
8  from generera_övergångsmatris import hämta_T
9

```

```

10 @njit
11 def viterbi_step(N, in_states, in_probs, v_prev, emit):
12     v_new = np.zeros(N)
13     ptr = np.zeros(N, dtype=np.int32)
14
15     for j in range(N):
16         prev_states_j = in_states[j]
17         probs_j = in_probs[j]
18
19         best_val = -1.0
20         best_l = 0
21
22         for k in range(len(prev_states_j)):
23             l = prev_states_j[k]
24             val = v_prev[l] * probs_j[k]
25             if val > best_val:
26                 best_val = val
27                 best_l = k
28
29         v_new[j] = best_val * emit[j]
30         ptr[j] = best_l
31
32     return v_new, ptr
33
34
35 # =====
36 # DOLD MARKOVKEDJA (HIDDEN MARKOV MODEL)
37 # =====
38 class DoldMarkovkedja(Markovkedja):
39     """
40     En dold Markovkedja (Hidden Markov Model, HMM).
41
42     Funktionalitet:
43     • simulerar dolda tillstånd baserat på Markovkedjan (via basklassen)
44     • genererar observationer bitvis (0, 1 eller None)
45     • beräknar:
46         - emissionssannolikheter (vektoriserat)
47         - forward (filtrering)
48         - backward (smoothing)
49         - forward-backward (posterior)
50         - Viterbi (MAP-sekvens)
51
52     Optimeringar:
53     • använder sparse CSR-matris från Markovkedja
54     • använder förberäknad TT (transponatet) för snabb forward
55     • emissioner beräknas vektoriserat över alla tillstånd
56     • Viterbi använder endast icke-noll-övergångar
57     """
58
59 # -----
60 # Konstruktor
61 # -----
62 def __init__(self, tillstand, overgangsmatris, p_emission):
63     """
64     Parametrar
65     -----
66     tillstand : list[tuple[int]]

```

```

67         Lista av binära dolda tillstånd.
68         overgangsmatrix : csr_matrix
69         Gles (CSR) övergångsmatrix för Markovkedjan.
70         p_emission : float eller array
71         Sannolikhet att observera rätt bit.
72         Om float → samma för alla bitar.
73         Om array → en sannolikhet per bit.
74         """
75
76         # Initiera basklassen (Markovkedja)
77         super().__init__(tillstand, overgangsmatrix)
78
79         self.n_bits = len(self.tillstand[0])
80
81         # Emissionssannolikheter
82         if isinstance(p_emission, (float, int)):
83             self.p_emit = np.full(self.n_bits, float(p_emission))
84         else:
85             self.p_emit = np.array(p_emission, dtype=float)
86
87         # Tillstånd som NumPy-matrix: form (N × n_bits)
88         # Används för snabb maskmatchning i emissioner
89         self.state_matrix = np.array(self.tillstand, dtype=int)
90
91         # Förberäkna inkommande övergångar l→j i Numba-typed lists
92         in_states = List()
93         in_probs = List()
94
95         for j in range(self.N):
96             start = self.T_T.indptr[j]
97             end = self.T_T.indptr[j+1]
98
99             # Varje element är en vanlig NumPy-array
100             in_states.append(self.T_T.indices[start:end].astype(np.int32))
101             in_probs.append(self.T_T.data[start:end].astype(np.float64))
102
103         self.in_states = in_states
104         self.in_probs = in_probs
105
106
107         # -----
108         # Emissionsprocess (simulering)
109         # -----
110         def emit(self, dolt_tillstand):
111             """
112             Genererar en observation givet ett dolt tillstånd.
113
114             För varje bit:
115             • korrekt bit med sannolikhet p_emit[k]
116             • annars None (saknad observation)
117             """
118             rand = np.random.rand(self.n_bits)
119             obs = []
120
121             for k in range(self.n_bits):
122                 if rand[k] < self.p_emit[k]:
123                     obs.append(dolt_tillstand[k])

```

```

124         else:
125             obs.append(None)
126
127         return tuple(obs)
128
129     # -----
130     # Simulering: dolda tillstånd + observationer
131     # -----
132     def simulera_dolda_index(self, steg, p_start=0.4):
133         """
134         Returnerar:
135             start_index : index för starttillståndet
136             seq_index   : lista av index för dolda tillstånd
137         """
138         return super().simulera_index(steg, p_start)
139
140
141     def simulera_dolda(self, steg, p_start=0.4):
142         start_idx, seq_idx = self.simulera_dolda_index(steg, p_start)
143         return [self.index_till_tillstand(i) for i in seq_idx]
144
145
146     def simulera_observationer(self, steg, p_start=0.4):
147         dolda = self.simulera_dolda(steg, p_start)
148         obs = [self.emit(s) for s in dolda]
149         return dolda, obs
150
151     # -----
152     # Emissionssannolikhet (vektoriserad över alla tillstånd)
153     # -----
154     def emissionsvektor(self, observation):
155         """
156         Returnerar en vektor  $p$  av längd  $N$  där
157              $p[i] = P(\text{observation} \mid \text{tillstånd } i)$ .
158
159         För varje bit:
160             • om None: påverkar ej sannolikheten
161             • om obs = 0/1: maska bort tillstånd som ej matchar
162         """
163         p = np.ones(self.N)
164
165         for k, obs in enumerate(observation):
166             if obs is None:
167                 continue
168
169             # match[k] = True där tillståndens bit k stämmer med observationen
170             match = (self.state_matrix[:, k] == obs)
171
172             # fel tillstånd → sannolikhet 0
173             p[~match] = 0.0
174
175             # rätt tillstånd → multiplicera med p_emit
176             p[match] *= self.p_emit[k]
177
178         return p
179
180     # -----

```

```

181 # Forward-algoritmen
182 # -----
183 def forward(self, observationer, progbar):
184     """
185     Forward-pass:
186     fwd[t] = normalize( emissions[t] * (TT @ fwd[t-1]) )
187     """
188     L = len(observationer)
189     fwd = np.zeros((L, self.N))
190
191     # t=0
192     emit0 = self.emissionsvektor(observationer[0])
193     fwd[0] = self.start_förd * emit0
194     fwd[0] /= fwd[0].sum()
195
196
197     # t>0
198     iterator = range(1,L)
199     ↪ # progbar = False ingen progbar skapas
200     if progbar == None:
201         iterator = tqdm(range(L), desc="Forward") if self.n_bits >= 16 else
202         ↪ range(1,L) # progbar = None progbar skapas om n>= 16 detta
203         ↪ tal går att ändra om man vill ha en annan default
204     if progbar:
205         iterator = tqdm(range(L), desc="Forward")
206         ↪ # progbar = True skapar progbar oavsett
207
208     for t in iterator:
209         emit = self.emissionsvektor(observationer[t])
210         fwd[t] = emit * (self.T_T.dot(fwd[t-1]))
211         s = fwd[t].sum()
212         if s > 0:
213             fwd[t] /= s
214
215     return fwd
216
217 # -----
218 # Backward-algoritmen
219 # -----
220 def backward(self, observationer, progbar):
221     """
222     Backward-pass:
223     bwd[t] = normalize( T @ (emissions[t+1] * bwd[t+1]) )
224     """
225     L = len(observationer)
226     bwd = np.zeros((L, self.N))
227     bwd[-1] = 1.0
228
229     iterator = range(L - 2, -1, -1)
230     ↪ # progbar = False ingen progbar skapas
231     if progbar == None:
232         iterator = tqdm(range(L - 2, -1, -1), desc="Backward") if self.n_bits
233         ↪ >= 16 else range(L - 2, -1, -1) # progbar = None progbar skapas
234         ↪ om n>= 16 detta tal går att ändra om man vill ha en annan
235         ↪ default
236     if progbar:

```

```

229         iterator = tqdm(range(L - 2, -1, -1), desc="Backward")
230         ↪ # progbar = True skapar progbar oavsett
231     for t in iterator:
232         emit = self.emissionsvektor(observationer[t+1])
233         bwd[t] = self.T.dot(emit * bwd[t+1])
234         s = bwd[t].sum()
235         if s > 0:
236             bwd[t] /= s
237
238     return bwd
239
240 # -----
241 # Forward-backward (posterior)
242 # -----
243 def forward_backward(self, observationer, progbar = None):
244     """
245     Returnerar posteriora sannolikheter:
246     posterior[t,i] = P(X_t = i | alla observationer)
247     """
248     fwd = self.forward(observationer, progbar)
249     bwd = self.backward(observationer, progbar)
250
251     post = fwd * bwd
252     post /= post.sum(axis=1, keepdims=True)
253     return post
254
255 # -----
256 # Viterbi
257 # -----
258 def viterbi(self, observationer, progbar = None):
259     """
260     Viterbi-algoritmen (MAP-sekvens):
261     Beräknar den mest sannolika följden av dolda tillstånd.
262     """
263     L = len(observationer) # Antalet steg. Är N
264     ↪ i pseudokoden
265     v = np.zeros((L, self.N)) # Motsvarar delta
266     ptr = np.zeros((L, self.N), dtype=int) # Motsvarar psi
267
268     # Initialisering
269     v[0] = self.start_förd * self.emissionsvektor(observationer[0]) #
270     ↪ delta_k(1) = pi_k * P(y_1 | X_1=k), här antar vi implicit
271
272     # Rekursion
273
274     iterator = range(1,L)
275     ↪ # progbar = False ingen progbar skapas
276     if progbar == None:
277         iterator = tqdm(range(L), desc="Viterbi") if self.n_bits >= 16 else
278         ↪ range(1,L) # progbar = None progbar skapas om n >= 16
279     if progbar:
280         iterator = tqdm(range(L), desc="Viterbi")
281         ↪ # progbar = True skapar progbar oavsett
282
283     for t in iterator: # För varje tidssteg.
284         ↪ t motsvarar i

```

```

278         emit = self.emissionsvektor(observationer[t]) # Lista av alla
                ↪ emissionssannolikheter  $P(y_t/X_t=j)$  för alla  $j$ 

279
280         # OPTIMERAD DEL:
281         # Här gör vi hela rekursionen för  $\delta_j(t)$  och  $\pi_j(t)$ 
282         # med ett enda numba-anrop istället för en Python-loop över  $j$ .
283         #
284         # Matematiskt beräknas:
285         #  $\delta_j(t) = \max_l [\delta_l(t-1) p(l+j)] * P(y_t | j)$ 
286         #  $\pi_j(t) = \operatorname{argmax}_l [\delta_l(t-1) p(l+j)]$ 
287
288         v[t], ptr[t] = viterbi_step(
289             self.N,
290             self.in_states,
291             self.in_probs,
292             v[t-1],
293             emit
294         )
295
296
297         # Terminering
298         last = int(np.argmax(v[-1])) # Ger det index  $k$  som
                ↪ ger max i  $\delta_k(N)$ . Last =  $X*_N$ 
299         sekvens = [last] # Skapa listan med
                ↪ alla  $X*_t$  som nu endast innehåller  $X*_N$ 
300
301         # Tillbakasprång
302         for t in range(L-1, 0, -1):
303             sekvens.insert(0, ptr[t, sekvens[0]]) #  $X*_{t-1} =$ 
                ↪  $\pi_t(X*_t)$ 
304
305         return [self.index_till_tillstand(i) for i in sekvens] # Omvandlar alla
                ↪  $X*_t$  till tillstånd och inte index
306

```

A.4 Tester

Här är alla tester som använts i studien.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from tqdm import tqdm
4  import json
5  import seaborn as sns
6
7  # IMPORTS
8  from generera_övergångsmatris import hämta_T
9  from dold_markovkedja import DoldMarkovkedja
10 from markovkedja import Markovkedja
11
12
13 # =====
14 # 1. METRIKER
15 # =====
16
17 def andel_rätt_hamming(true_seq, est_seq):
18     n = len(true_seq[0])
19     return [

```

```

20         1 - sum(a != b for a, b in zip(t, v)) / n
21         for t, v in zip(true_seq, est_seq)
22     ]
23
24
25 def andel_rätt_tillstånd(true_seq, est_seq):
26     return [1 if t == v else 0 for t, v in zip(true_seq, est_seq)]
27
28
29 def andel_fel_fb(true_seq, posterior, tillstånd):
30     n = len(tillstånd[0])
31     fb_seq = fb_posterior_till_kedja(posterior, tillstånd)
32
33     return [
34         sum(abs(a - b) for a, b in zip(t, v))
35         for t, v in zip(true_seq, fb_seq)
36     ]
37
38
39 # =====
40 # 2. FB-HJÄLPFUNKTIONER
41 # =====
42
43 def fb_posterior_till_kedja(posterior, tillstånd):
44     state_matrix = np.array(tillstånd, dtype=float)
45     return [tuple(posterior[t] @ state_matrix) for t in
46             ↪ range(posterior.shape[0])]
47
48 # =====
49 # 3. BAS-TESTER
50 # =====
51
52 def test_fb(n, smittotid, utskrivningstid,
53            steg=15, simuleringar=100, p_start=0.4, p=0.5):
54
55     tillstånd, T = hämta_T(n, smittotid, utskrivningstid)
56     H = DoldMarkovkedja(tillstånd, T, p)
57
58     total_fel = 0
59
60     for _ in tqdm(range(simuleringar), desc = "ett p"):
61         dolda, obs = H.simulera_observationer(steg, p_start=p_start)
62         posterior = H.forward_backward(obs)
63
64         total_fel += sum(andel_fel_fb(dolda, posterior, tillstånd))
65
66     total = simuleringar * steg * n
67     return 1 - total_fel / total
68
69
70 def test_viterbi(n, smittotid, utskrivningstid,
71                 steg=15, simuleringar=100, p_start=0.4, p=0.5):
72
73     tillstånd, T = hämta_T(n, smittotid, utskrivningstid)
74     H = DoldMarkovkedja(tillstånd, T, p)
75

```

```

76     biträtt = 0
77     tillstånds rätt = 0
78
79     for _ in tqdm(range(simuleringar), desc = "ett värde"):
80         dolda, obs = H.simulera_observationer(steg, p_start=p_start)
81         vit = H.viterbi(obs)
82
83         biträtt += sum(andel_rätt_hamming(dolda, vit))
84         tillstånds rätt += sum(andel_rätt_tillstånd(dolda, vit))
85
86     norm = simuleringar * (steg + 1)
87     return biträtt / norm, tillstånds rätt / norm
88
89
90 # =====
91 # 4. SWEEP-EXPERIMENT
92 # =====
93
94 def sweep_fb(n, smittotid, utskrivningstid,
95             steg, antal_steg_p, simuleringar, p_start,
96             spara_fil=None,
97             plotta=True):
98
99     p_values = []
100     noggrannhet = []
101
102     for k in tqdm(range(antal_steg_p + 1), desc="p-sweep"):
103         p = k / antal_steg_p
104         p_values.append(p)
105
106         noggrannhet.append(
107             test_fb(n, smittotid, utskrivningstid,
108                    steg=steg,
109                    simuleringar=simuleringar,
110                    p_start=p_start,
111                    p=p)
112         )
113
114     # =====
115     # SPARA
116     # =====
117     if spara_fil is not None:
118         spara_resultat_fb(spara_fil, p_values, noggrannhet)
119
120     # =====
121     # PLOTTA
122     # =====
123     if plotta:
124         plot_resultat(p_values, noggrannhet)
125
126     return p_values, noggrannhet
127
128
129
130 def sweep_viterbi(n, smittotid, utskrivningstid,
131                  steg, antal_steg_p, simuleringar, p_start,
132                  spara_fil=None,

```

```

133         plotta=True):
134
135     p_values = []
136     bit_noggrannhet = []
137     tillstånd_noggrannhet = []
138
139     for k in tqdm(range(antal_steg_p + 1), desc="p-sweep Viterbi"):
140         p = k / antal_steg_p
141         p_values.append(p)
142
143         bit, tillst = test_viterbi(
144             n, smittotid, utskrivningstid,
145             steg=steg,
146             simuleringar=simuleringar,
147             p_start=p_start,
148             p=p
149         )
150
151         bit_noggrannhet.append(bit)
152         tillstånd_noggrannhet.append(tillst)
153
154         # =====
155         # SPARA
156         # =====
157         if spara_fil is not None:
158             spara_resultat_vit(
159                 spara_fil,
160                 p_values,
161                 bit_noggrannhet,
162                 tillstånd_noggrannhet
163             )
164
165         # =====
166         # PLOTTA
167         # =====
168         if plotta:
169             plot_resultat(p_values, bit_noggrannhet)
170             plot_resultat(p_values, tillstånd_noggrannhet)
171
172     return p_values, bit_noggrannhet, tillstånd_noggrannhet
173
174
175 def sweep_tidsförhållande(n,
176                           utskrivningstid=1,
177                           antal_punkter=41,
178                           simuleringar=100,
179                           steg=100,
180                           p_start=0.4,
181                           p_emit=0.4,
182                           spara_fil=None,
183                           plotta=True):
184
185     smittotider = np.logspace(-6, 6, antal_punkter, base=2)
186
187     acc_bitar = []
188     acc_tillstånd = []
189

```

```

190     for ts in tqdm(smittotider, desc="smittotid sweep"):
191
192         bit, tillst = test_viterbi(
193             n=n,
194             smittotid=ts,
195             utskrivningstid=utskrivningstid,
196             steg=steg,
197             simuleringar=simuleringar,
198             p_start=p_start,
199             p=p_emit
200         )
201
202         acc_bitar.append(bit)
203
204
205     # =====
206     # SPARA
207     # =====
208     if spara_fil is not None:
209         with open(spara_fil, "w") as f:
210             json.dump({
211                 "smittotider": list(smittotider),
212                 "acc_bitar": acc_bitar
213             }, f)
214
215     # =====
216     # PLOTTA
217     # =====
218     if plotta:
219         plot_viterbi_linje(smittotider, acc_bitar)
220
221     return smittotider, acc_bitar
222
223
224     # =====
225     # 5. SPARA / LADDA
226     # =====
227
228     def spara_resultat_fb(filnamn, p_values, noggrannhet):
229         with open(filnamn, "w") as f:
230             json.dump({"p": p_values, "noggrannhet": noggrannhet}, f)
231
232
233     def ladda_resultat_fb(filnamn):
234         with open(filnamn) as f:
235             data = json.load(f)
236             return data["p"], data["noggrannhet"]
237
238
239     def spara_resultat_vit(filnamn, p_values, bit, tillstånd):
240         with open(filnamn, "w") as f:
241             json.dump({"p": p_values, "bit": bit, "tillstånd": tillstånd}, f)
242
243
244     def ladda_resultat_vit(filnamn):
245         with open(filnamn) as f:
246             data = json.load(f)

```

```

247     return data["p"], data["bit"], data["tillstånd"]
248
249
250 # =====
251 # 6. PLOTTNING
252 # =====
253
254 def hitta_tröskel(p_values, y_values, tröskel):
255     for p, y in zip(p_values, y_values):
256         if y >= tröskel:
257             return p, y
258     return None, None
259
260
261 def plot_resultat(p_values, y_values, tröskel=0.95):
262     p_star, y_star = hitta_tröskel(p_values, y_values, tröskel)
263
264     plt.figure(figsize=(8, 5))
265     plt.plot(p_values, y_values, marker="o")
266
267     plt.axhline(tröskel, color = "red", linestyle="--")
268     plt.text(0.05, tröskel + 0.03, f"y = {tröskel}", color="red")
269     if p_star is not None:
270         plt.axvline(p_star, linestyle="--")
271         plt.plot(p_star, y_star, "ko")
272         plt.text(p_star + 0.02, 0.1, f"p = {p_star:.2f}", color="blue")
273
274     plt.xlim(0, 1)
275     plt.ylim(0, 1.05)
276     plt.xlabel(r"Testsannolikhet, $p$")
277     plt.ylabel(r"Noggrannhet")
278     plt.tight_layout()
279     plt.show()
280
281
282 def plot_fb_från_fil(filnamn, tröskel=0.95):
283     p_values, y_values = ladda_resultat_fb(filnamn)
284     plot_resultat(p_values, y_values, tröskel=tröskel)
285
286 def plot_viterbi_fran_fil(filnamn, tröskel=0.95):
287     p, bit, tillstånd = ladda_resultat_vit(filnamn)
288     plot_resultat(p, bit, tröskel)
289     plot_resultat(p, tillstånd, tröskel)
290
291
292
293 def plot_viterbi_linje(smittotider, acc_bitar):
294
295     plt.figure(figsize=(7,4))
296
297     plt.plot(smittotider, acc_bitar, marker='o')
298
299     # log-skala
300     plt.xscale("log")
301     idx = acc_bitar.index(min(acc_bitar))
302     tid = smittotider[idx]
303     plt.axvline(tid)

```

```

304     plt.xlabel(r"$\frac{1}{\lambda}$")
305     plt.ylabel("Noggrannhet")
306
307
308     plt.tight_layout()
309     plt.show()
310
311 def plot_viterbi_tidsförhållande_fran_fil(filnamn):
312     with open(filnamn) as f:
313         data = json.load(f)
314
315     smittotider = data["smittotider"]
316     acc_bitar = data["acc_bitar"]
317     plot_viterbi_linje(smittotider, acc_bitar)
318
319
320 def plot_förankring(smittotider, resultat, tröskel = 0):
321     res = [-a for a in resultat]
322     tid_star, resultat_star = hitta_tröskel(smittotider, res, tröskel)
323     plt.figure(figsize=(7,4))
324     plt.plot(smittotider, resultat, linewidth = 4)
325
326     # log x-axel
327     plt.xscale("log")
328
329     # streckad linje vid 0
330     plt.axhline(0, linestyle="--", color = "red")
331
332
333     if tid_star is not None:
334         plt.axvline(tid_star, linestyle="--")
335         plt.plot(tid_star, resultat_star, "ko")
336         plt.text(tid_star + 0.5, 0.1, f"c = {tid_star:.2f}", color="black")
337
338     plt.xlabel(r"$\frac{1}{\lambda}$")
339     plt.ylabel(r"Fastnamått")
340
341     plt.tight_layout()
342     plt.show()
343
344 def plot_förankring_fran_fil(filnamn):
345
346     with open(filnamn) as f:
347         data = json.load(f)
348
349     smittotider = data["smittotider"]
350     resultat = data["resultat"]
351
352     plot_förankring(smittotider, resultat)
353
354
355 # =====
356 # 7. HEATMAPS
357 # =====
358
359 def heatmaps_full(n, smittotid, utskrivningstid,
360                  p_emit, steg, p_start=0.4):

```

```

361
362     tillstånd, T = hämta_T(n, smittotid, utskrivningstid)
363     H = DoldMarkovkedja(tillstånd, T, p_emit)
364
365     dolda, obs = H.simulera_observationer(steg, p_start=p_start)
366     posterior = H.forward_backward(obs)
367     viterbi = H.viterbi(obs)
368
369     state_matrix = np.array(tillstånd, dtype=float)
370     fb_mean = np.array([posterior[t] @ state_matrix for t in range(steg)])
371
372     dolda = np.array(dolda)
373     viterbi = np.array(viterbi)
374
375     # ===== Dolda =====
376     plt.figure(figsize=(6,4))
377
378     ax = sns.heatmap(
379         dolda,
380         cmap="Blues",
381         vmin=0,
382         vmax=1,
383         annot=True,
384         fmt=".0f",
385         cbar=False
386     )
387     ax.invert_yaxis()
388     ax.set_yticklabels(ax.get_yticklabels(), rotation=0)
389     ax.set_xticklabels(range(1, len(dolda[0]) + 1))
390
391     # röda cirklar där obs finns
392     for i in range(len(obs)):
393         for j in range(len(obs[0])):
394             if obs[i][j] is not None:
395                 ax.scatter(
396                     j + 0.5,
397                     i + 0.5,
398                     s=200,
399                     facecolors='none',
400                     edgecolors='red',
401                     linewidths=2
402                 )
403
404     plt.xlabel("Sängplats")
405     plt.ylabel("Tid")
406     plt.tight_layout()
407     plt.show()
408
409     # ===== FB =====
410     plt.figure(figsize=(6,4))
411     ax = sns.heatmap(fb_mean, cmap="PuBu", vmin=0, vmax=1,
412                     annot=True, fmt=".2f")
413     ax.invert_yaxis()
414     ax.set_yticklabels(ax.get_yticklabels(), rotation=0)
415     ax.set_xticklabels(range(1, len(dolda[0]) + 1))
416     plt.xlabel("Sängplats")
417     plt.ylabel("Tid")

```

```

418     plt.tight_layout()
419     plt.show()
420
421     # ===== Viterbi =====
422     plt.figure(figsize=(6,4))
423     ax = sns.heatmap(viterbi, cmap="PuBu", vmin=0, vmax=1,
424                     annot=True, fmt=".0f", cbar=False)
425     ax.invert_yaxis()
426     ax.set_yticklabels(ax.get_yticklabels(), rotation=0)
427     ax.set_xticklabels(range(1, len(dolda[0]) + 1))
428     plt.xlabel("Sängplats")
429     plt.ylabel("Tid")
430     plt.tight_layout()
431     plt.show()
432
433
434     # =====
435     # 8. FÖRANKRINGSTEST
436     # =====
437
438     def beräkna_förankringsscore(states, T, n,
439                                   p_start=0.4,
440                                   steg=100):
441
442         mk = Markovkedja(states, T)
443
444         start_idx, seq_index = mk.simulera_index(
445             steg=steg,
446             p_start=p_start
447         )
448
449         smitt = np.array([sum(states[i]) for i in seq_index])
450
451         zero_hits = np.where(smitt == 0)[0]
452         if len(zero_hits) > 0:
453             t = zero_hits[0]
454             return -(1 - t / steg)
455
456         full = n
457         full_hits = np.where(smitt == full)[0]
458
459         if len(full_hits) == 0:
460             return 0.0
461
462         t0 = full_hits[0]
463
464         studs = 0
465         last = True
466
467         for t in range(t0+1, len(smitt)):
468             if smitt[t] == full:
469                 if not last:
470                     studs += 1
471                 last = True
472             else:
473                 last = False
474

```

```

475         if studs >= 3:
476             return +(1 - t0 / steg)
477
478     return 0.0
479
480
481 def sweep_förankring(n,
482                     utskrivningstid=1,
483                     antal_punkter=41,
484                     simuleringar=50,
485                     steg=100,
486                     p_start=0.4,
487                     spara_fil=None,
488                     plotta=True):
489
490     smittotider = np.logspace(-5, 15, antal_punkter, base=2)
491     resultat = []
492
493     for ts in tqdm(smittotider, desc="smittotid sweep"):
494
495         states, T = hämta_T(n, ts, utskrivningstid)
496
497         scores = [
498             beräkna_förankringsscore(
499                 states, T, n,
500                 p_start=p_start,
501                 steg=steg
502             )
503             for _ in range(simuleringar)
504         ]
505
506         resultat.append(np.mean(scores))
507
508         # =====
509         # SPARA
510         # =====
511         if spara_fil is not None:
512             with open(spara_fil, "w") as f:
513                 json.dump({
514                     "smittotider": list(smittotider),
515                     "resultat": resultat
516                 }, f)
517
518         # =====
519         # PLOTTA
520         # =====
521         if plotta:
522             plot_förankring(smittotider, resultat)
523
524     return smittotider, resultat
525
526
527 # =====
528 # 9. MAIN (KÖR TESTER)
529 # =====
530
531 if __name__ == "__main__":

```

```

532
533     # =====
534     # Grundparametrar
535     # =====
536     n = 10
537     smittotid = 1
538     utskrivningstid = 4
539     steg = 50
540     simuleringar = 10
541     antal_steg_p = 30
542     p_start = 0.4
543
544     # =====
545     # VÄLJ VILKA TESTER SOM SKA KÖRAS
546     # =====
547     kör_fb = False
548     kör_viterbi_p = False
549     kör_viterbi_tid = False
550     kör_förankring = False
551     kör_heatmap = True
552
553     # =====
554     # FB-sweep (över p)
555     # =====
556     if kör_fb:
557         sweep_fb(
558             n, smittotid, utskrivningstid,
559             steg, antal_steg_p, simuleringar, p_start,
560             spara_fil="fb.json",
561             plotta=True
562         )
563
564     # =====
565     # Viterbi (över p)
566     # =====
567     if kör_viterbi_p:
568         sweep_viterbi(
569             n, smittotid, utskrivningstid,
570             steg, antal_steg_p, simuleringar, p_start,
571             spara_fil="vit.json",
572             plotta=True
573         )
574
575     # =====
576     # Viterbi (över smittotid)
577     # =====
578
579     if kör_viterbi_tid:
580         sweep_tidsförhållande(
581             n=n,
582             utskrivningstid=1,
583             antal_punkter=41,
584             simuleringar=simuleringar,
585             steg=steg,
586             p_start=p_start,
587             p_emit=0.4,
588             spara_fil="viterbi_tid.json",

```

```

589         plotta=True
590     )
591
592
593     # =====
594     # Förankring
595     # =====
596
597     if kör_förankring:
598         sweep_förankring(
599             n=n,
600             utskrivningstid=1,
601             antal_punkter=41,
602             simuleringar=simuleringar,
603             steg=steg,
604             p_start=p_start,
605             spara_fil="förankring.json",
606             plotta=True
607         )
608
609
610     # =====
611     # Heatmaps (en simulering)
612     # =====
613     if kör_heatmap:
614         heatmaps_full(
615             n=n,
616             smittotid=1,
617             utskrivningstid=2,
618             p_emit=0.4,
619             steg=10,
620             p_start=p_start
621         )
622
623
624
625     plot_fb_från_fil("fb.json")
626     plot_viterbi_fran_fil("vit.json")
627     plot_viterbi_tidsförhållande_fran_fil("viterbi_tid.json")
628     plot_förankring_fran_fil("förankring.json")

```